



**POLITÉCNICO
DE LEIRIA**

ESCOLA SUPERIOR
DE TECNOLOGIA
E GESTÃO

SBOM Lens: A Graph-Based Framework for Software Supply Chain Risk Assessment

Structural Metrics and Network Topology
Analysis of Software Dependency Networks

Igor Ferreira do Nascimento

School of Management and Technology
Department of Computer Engineering
Master in Cybersecurity & Digital Forensics

Leiria, May 2026



**POLITÉCNICO
DE LEIRIA**

ESCOLA SUPERIOR
DE TECNOLOGIA
E GESTÃO

SBOM Lens: A Graph-Based Framework for Software Supply Chain Risk Assessment

Structural Metrics and Network Topology
Analysis of Software Dependency Networks

Igor Ferreira do Nascimento

Student No. 2240253

Supervisor: João da Silva Pereira

Full Professor, Polytechnic of Leiria

School of Management and Technology
Department of Computer Engineering
Master in Cybersecurity & Digital Forensics

Project

Leiria, May 2026

SBOM Lens: A Graph-Based Framework for Software Supply Chain Risk Assessment

Copyright © 2026 - Igor Ferreira do Nascimento, School of Management and Technology.

This dissertation is original work, written solely for this purpose, and all the authors whose studies and publications contributed to it have been duly cited. Partial reproduction is allowed with acknowledgment of the author and reference to the degree, academic year, institution—*Polytechnic University of Leiria*—and public defense date.



Preparation of this work was facilitated by the use of the *IPLeiria-Thesis* template.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Professor João da Silva Pereira, for his invaluable guidance, support, and encouragement throughout the course of this project. Also i would like to thank my college professor Daniel Ratton for introducing me to the world of network science, which was fundamental for the development of this project. I am also grateful to my colleagues and friends who provided encouragement and support during the research process. Finally, I would like to thank my family for their support, in particular my grandma and my father-in-law, who always belived in me and encouraged me to pursue my academic goals. This project would not have been possible without the support and encouragement of all these people, and I am deeply grateful for their presence in my life and for been the reason I am where I am.

Resumo

A crescente adoção de componentes de software de terceiros aumenta a superfície de ataque das cadeias de fornecimento, tornando difícil para as organizações compreender e mitigar os riscos introduzidos pelas suas dependências. O Software Bill of Materials (SBOM) emerge como um mecanismo de transparência, mas os formatos existentes carecem de ferramentas analíticas que transformem dados de inventário em inteligência de risco acionável.

Esta dissertação apresenta o SBOM Lens, uma framework baseada em grafos para avaliação de risco em cadeias de fornecimento de software. O sistema ingere SBOMs no formato CycloneDX e constrói um grafo de dependências orientado, sobre o qual aplica técnicas de ciência de redes para expor vulnerabilidades estruturais. O trabalho cobre quatro domínios principais: (1) a visualização interativa do grafo SBOM, que permite explorar e editar a topologia das dependências; (2) a gestão de vulnerabilidades, com integração em múltiplas fontes de dados como NVD, OSV e GitHub Security Advisories, incluindo o cálculo de scores CVSS v3.1; (3) a avaliação de risco estrutural, que identifica pontos de falha única, dependências circulares, conflitos de versão e concentração de dependências através de métricas de entropia e decomposição em núcleos; e (4) a análise de grafos, que aplica algoritmos de detecção de comunidades, decomposição k-core e análise espectral para modelar a propagação de vulnerabilidades.

O SBOM Lens demonstra que a teoria dos grafos e a ciência de redes constituem uma abordagem eficaz para priorizar riscos de segurança em cadeias de fornecimento de software modernas.

Palavras-Chave: SBOM, Segurança de Cadeias de Fornecimento de Software, Teoria dos Grafos, Análise de Vulnerabilidades, Avaliação de Risco, CycloneDX.

Abstract

The widespread adoption of third-party software components continuously expands the attack surface of modern software supply chains, making it difficult for organisations to understand and mitigate the risks introduced by their dependencies. The Software Bill of Materials (SBOM) has emerged as a transparency mechanism, yet existing tooling largely treats SBOMs as flat inventories rather than structured graphs amenable to rigorous analysis.

This dissertation presents SBOM Lens, a graph-based framework for software supply chain risk assessment. The system ingests SBOMs in CycloneDX format and constructs a directed dependency graph, over which network science techniques are applied to expose structural vulnerabilities. The work covers four main domains: (1) interactive SBOM graph visualisation, enabling exploration and editing of the dependency topology; (2) vulnerability management, integrating multiple data sources — NVD, OSV, and GitHub Security Advisories — with full CVSS v3.1 base score computation; (3) structural risk assessment, which identifies single points of failure via articulation point analysis, circular dependencies, version conflicts through diamond dependency detection, and dependency concentration using entropy metrics and k-core decomposition; and (4) graph analysis, applying community detection, k-core decomposition, and spectral graph theory to model vulnerability propagation using epidemic spreading analogues.

SBOM Lens demonstrates that graph theory and network science provide an effective foundation for contextualising and prioritising security risk in modern software supply chains.

Keywords: SBOM, Software Supply Chain Security, Graph Theory, Vulnerability Analysis, Risk Assessment, CycloneDX.

AI Acknowledgement

I acknowledge the use of Claude (<https://claude.ai>) to refine the academic tone and improve the linguistic accuracy of this work, including aspects of grammar, punctuation, and vocabulary.

Contents

<i>List of Figures</i>	xiii
<i>List of Tables</i>	xv
<i>Glossary</i>	xvii
<i>Acronyms</i>	xix
<i>Symbols</i>	xxi
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	3
1.3 Research Objectives	4
1.4 Contributions	5
1.5 Scope and Limitations	5
1.6 Thesis Structure	6
2 Background	8
2.1 Software Bill of Materials and CycloneDX	8
2.1.1 What is a Software Bill of Materials?	8
2.1.2 SBOM Format Landscape	9
2.1.3 CycloneDX JSON Structure and SBOM Lens Fields	10
2.2 Software Supply Chain Security	11
2.2.1 The Software Supply Chain	11
2.2.2 Attack Taxonomy	11
2.2.3 Policy and Standards Response	12
2.3 Graph Theory Fundamentals	13
2.3.1 Directed Graphs and Basic Definitions	13
2.3.2 Directed Acyclic Graphs and Cycles	13
2.3.3 Connectivity and Reachability	14
2.3.4 Articulation Points and Bridge Edges	14
2.4 Network Science Concepts	15
2.4.1 Scale-Free Networks and Power-Law Degree Distributions	15
2.4.2 Small-World Networks	16
2.4.3 Centrality Measures	16

2.4.4	Community Detection and Modularity	17
2.4.5	K-Core Decomposition	17
2.4.6	Propagation Modelling — Future Directions	18
2.5	Vulnerability Databases	18
2.5.1	CVE and the National Vulnerability Database	19
2.5.2	Open Source Vulnerabilities (OSV)	19
2.5.3	GitHub Security Advisories	20
2.5.4	CVSS v3.1 Scoring System	20
3	Related Work	22
3.1	Existing SBOM Analysis Tools	22
3.2	Graph-Based Security Research	24
3.2.1	Dependency Graph Analysis	24
3.2.2	Software Vulnerability Graphs	24
3.2.3	Call Graph and Static Analysis	24
3.2.4	Network Science Applied to Software	24
3.3	Supply Chain Attack Case Studies	24
3.3.1	SolarWinds SUNBURST (December 2020)	24
3.3.2	log4shell (CVE-2021-44228, December 2021)	25
3.4	Gap Analysis	25
4	System Design	26
4.1	System Architecture	26
4.1.1	Overview and Design Rationale	26
4.1.2	Frontend Layer	27
4.1.3	Backend Layer	27
4.1.4	Communication: REST API	28
4.2	CycloneDX Parsing Pipeline	28
4.2.1	Ingestion and Deduplication	28
4.2.2	Component Extraction	29
4.2.3	Graph Construction from Dependencies	29
4.3	Graph Data Model	30
4.3.1	Node Representation	30
4.3.2	Edge Representation	31
4.3.3	In-Memory Store	31
4.4	SBOM Graph Visualisation	32
4.4.1	Graph Rendering with Cytoscape.js	32
4.4.2	Layout Algorithms	32
4.4.3	Core Interaction Model	33

5	Vulnerability Analysis	34
5.1	Multi-Source Vulnerability Integration	34
5.1.1	Rationale for Multi-Source Design	35
5.1.2	OSV: Primary Source	35
5.1.3	NVD Integration	36
5.1.4	GitHub Security Advisories	36
5.2	Deduplication and Storage	37
5.2.1	VulnStore Design	37
5.2.2	Deduplication Logic	38
5.3	CVSS v3.1 Base Score Computation	38
5.3.1	Formula and Metric Groups	39
5.3.2	Worked Example: Log4Shell	39
5.3.3	Custom Client-Side Implementation	40
5.4	Vulnerability Management Workflow	41
5.4.1	Automated Fetch on SBOM Load	41
5.4.2	Status Lifecycle and REST API	41
5.5	Graph Overlay Visualisation	42
5.5.1	Severity Colour Coding	42
5.5.2	Blast Radius Analysis	43
5.5.3	Dominator Tree Analysis	44

Appendices

Annexes

List of Figures

5.1	SBOM Lens graph overlay showing vulnerability severity colour coding. Nodes are coloured by their highest-severity CVE: red for CRITICAL, orange for HIGH, yellow for MEDIUM, and green-yellow for LOW. Unaffected nodes retain the default blue style.	43
-----	---	----

List of Tables

3.1	Comparison of SBOM analysis tools across four capability dimensions. SBOM Lens (last row) is the reference implementation evaluated in this thesis. . .	23
-----	---	----

1

Introduction

1.1 Motivation

Modern software is rarely built from scratch. The overwhelming majority of production systems depend on third-party open-source components: the Synopsys 2024 Open Source Security and Risk Analysis report found that 96% of commercial codebases contain open-source code and that the average codebase incorporates over 500 open-source components (**Synopsys2024OSSRA**). Each of these components introduces its own transitive closure of further dependencies, forming a supply chain whose attack surface is orders of magnitude larger than the first-party code that developers directly author. The *software supply chain* — the aggregate of all such components, their origins, build pipelines, and trust relationships — has consequently become one of the most consequential and least understood risk surfaces in modern cybersecurity.

The fragility of this supply chain was exposed in dramatic fashion by a succession of high-profile attacks. In December 2020, the SUNBURST campaign targeted SolarWinds, a network management vendor trusted by thousands of organisations including agencies of the United States federal government. Attackers compromised the vendor’s build pipeline and inserted malicious code into a signed software update that was subsequently distributed to approximately 18,000 customers (**CISA_AA20_352A**). The incident demonstrated how a single trusted upstream vendor can become an unwitting vector for cascading compromise across an entire downstream ecosystem. A year later, in December 2021, the disclosure of CVE-2021-44228 in Apache Log4j — universally known as *log4shell* — revealed a critical remote code execution vulnerability in a logging library present as a transitive dependency in an estimated three billion Java-based applications (**NIST_CVE_2021_44228**). The difficulty organisations faced in determining whether they were exposed illustrates precisely the challenge this thesis addresses: without a structural model of the dependency graph, the presence of a transitive dependency remains invisible until it is exploited. In March 2024, the *xz-utils* backdoor (CVE-2024-3094, CVSS 10.0) added a further dimension of supply chain

risk to the record: a patient, multi-year social engineering campaign against an open-source maintainer resulted in the insertion of a cryptographic backdoor into a core Linux compression utility that, had it reached widespread deployment, would have compromised SSH authentication on affected systems ([NVD_CVE_2024_3094](#)). Together, these three incidents establish that supply chain attacks are diverse in method, severe in potential impact, and increasing in sophistication.

Regulators around the world have responded by mandating greater transparency over software composition. In the European Union, the Cyber Resilience Act (CRA), entered into force in December 2024, enforces manufacturers and developers of products with digital elements sold in the EU market to document the components of their software and to address known vulnerabilities throughout the product lifecycle ([CRA2024](#)). The CRA explicitly names the SBOM — a machine-readable inventory of all software components and their dependency relationships — as a key instrument of this transparency obligation. In the United States, Executive Order 14028, signed in May 2021, similarly mandated that software supplied to the federal government be accompanied by an SBOM, defining it as “a formal record containing the details and supply chain relationships of various components used in building software” ([EO14028](#)). Executive Order 14144, issued in January 2025, reinforced and extended these requirements ([EO14144](#)). The SBOM has thus transitioned from a niche software engineering practice to a regulatory expectation on both sides of the Atlantic, and the volume of SBOMs in circulation is set to grow substantially in coming years.

Yet the emergence of the SBOM as a transparency artefact has not been matched by commensurate progress in analytical tooling. The dominant open-source tools in the SBOM ecosystem — Dependency-Track, grype, syft, and Sunshine/CycloneDX — treat SBOMs as flat component inventories ([DependencyTrack](#); [Grype](#); [Syft](#)). They answer two questions effectively: *what components are present?* and *do any of them have known CVEs?* What they do not do is model the *structure* of the dependency graph. When a vulnerability is disclosed in a given component, these tools report the affected component in isolation — but they provide no answer to the question a security analyst immediately needs: *which other components in my system depend on this one, directly or transitively, and are therefore also at risk?* Nor do they identify which components, if compromised, would expose the largest number of upstream dependents — all the components that transitively rely on them and would therefore inherit the risk. The result is that analysts must manually triage long flat lists of CVEs with no structural context to guide prioritisation.

The opportunity for a more rigorous approach lies in the observation that a software dependency graph is, by construction, a directed graph — and graph theory provides precisely the tools needed to answer both questions above. Given a directed dependency graph, the set of all components affected by a vulnerability in component v is the set of ancestors of v in the graph, computable in linear time. Graph-theoretic centrality measures further allow the identification of components that, if compromised, would affect a disproportionately large portion of the dependency tree — components that

a security analyst should treat as higher-priority regardless of their individual CVSS score. Articulation points identify components whose removal would disconnect the graph entirely, representing single points of structural failure. These techniques are well-established in graph theory (Cormen2009) but have not been systematically applied to SBOM dependency graphs.

This dissertation presents SBOM Lens, a graph-based framework for software supply chain risk assessment. The system ingests Software Bills of Materials in CycloneDX format and constructs a directed dependency graph, over which graph-theoretic techniques are applied to surface structural vulnerabilities not visible through flat-inventory analysis. The design, implementation, and evaluation of SBOM Lens are described in Chapter 4, Chapter 5, and ??, respectively, with empirical results presented in ??.

1.2 Problem Statement

Existing SBOM analysis tooling suffers from a structural limitation that is architectural rather than incidental: it treats the dependency graph as a flat list. When a security analyst queries a tool such as Dependency-Track or gype for the vulnerabilities present in a given SBOM, the result is a table of (component, CVE, severity) tuples ranked at most by CVSS score. There is no representation of the relationships between components, no indication of how many other components depend on a given vulnerable package, and no means of determining whether a vulnerable component sits at the centre of the dependency tree or at its periphery. This limitation has two concrete consequences for the analyst.

The first consequence is *invisible blast radius*. When a new vulnerability is disclosed in a widely-used library — as occurred with log4shell — the analyst’s immediate question is not “does this component have a CVE?” but rather “how much of my system is exposed?” Answering that question requires traversing the dependency graph upstream from the vulnerable component to find all packages that depend on it, directly or transitively. No mainstream SBOM tool performs this traversal; the analyst must do it manually, if they can do it at all.

The second consequence is *undifferentiated prioritisation*. A CVSS score measures the intrinsic severity of a vulnerability in isolation — attack vector, complexity, impact — but says nothing about the structural importance of the affected component within the system. A critical-severity CVE in a component that is a leaf node with no dependents poses a fundamentally different remediation burden than the same CVE in a component that is a shared transitive dependency of dozens of packages throughout the graph. Without graph-structural context, both appear equally urgent on a flat list, and the analyst has no principled basis for deciding where to act first.

The research gap is well-documented in the literature. A 2025 landscape study of SBOM tooling (SBOMlandscape2025) found that graph-based analysis of dependency structure is absent from the current tool ecosystem. The closest existing work is GUAC (Graph for Understanding Artifact Composition), developed under the OpenSSF um-

brella, which constructs provenance graphs across multiple projects and organisations (GUAC2024). GUAC addresses supply chain transparency at a cross-project, ecosystem-wide scale, but it does not perform within-SBOM graph analysis to assess the structural risk profile of a single software system. A detailed comparison of SBOM Lens with GUAC and other related tools is presented in [Chapter 3](#).

SBOM Lens addresses this gap by constructing a directed dependency graph from a CycloneDX SBOM and providing the analyst with two complementary views. The first is *impact traversal*: given any vulnerable component, the system immediately identifies all components that depend on it, at any depth, giving the analyst an accurate and complete picture of the blast radius. The second is *structural risk ranking*: using graph-theoretic measures, the system identifies which components are structurally most likely to propagate risk widely if compromised, allowing the analyst to prioritise monitoring and remediation based not only on CVE severity but on the component's position within the dependency graph. The research question is not whether these capabilities can be built — the system exists — but what they reveal when applied to real-world open-source SBOMs, a question answered empirically in ??.

1.3 Research Objectives

This work is framed as a design-and-analysis thesis: the system under study, SBOM Lens, has been built, and the objectives concern both its construction and what it reveals when applied empirically. Each objective maps directly to one of the implementation or evaluation chapters, allowing the conclusions drawn in ?? to be verified against each stated goal.

1. **Graph construction pipeline.** To design and implement a directed dependency graph construction pipeline that ingests CycloneDX JSON SBOMs and produces an annotated graph — with components as nodes and dependency relationships as directed edges — suitable for interactive visualisation and further graph-theoretic analysis. ([Chapter 4](#))
2. **Vulnerability integration.** To integrate multi-source vulnerability data from the National Vulnerability Database (NVD), the Open Source Vulnerabilities database (OSV), and GitHub Security Advisories, with full CVSS v3.1 base score computation, and to overlay the results on the dependency graph so that the analyst can immediately see which nodes carry known vulnerabilities and at what severity. ([Chapter 5](#))
3. **Graph-based structural risk analysis.** To apply a suite of graph-theoretic algorithms — including impact traversal to enumerate all components affected by a vulnerability in a given node, centrality and articulation point analysis to identify components with the highest structural propagation potential, and community detection to surface the modular structure of the dependency ecosystem — so as to give the analyst a principled basis for prioritising remediation beyond raw

CVSS scores. (??)

4. **Empirical evaluation.** To evaluate the framework against a set of real-world open-source project SBOMs and demonstrate, through graph-level and node-level metrics, that the approach surfaces structural risk information that is not available through flat-inventory analysis. (??)

1.4 Contributions

This work makes four contributions, each corresponding to a distinct system capability or analytical result.

1. **SBOM Graph Visualisation.** We design and implement an interactive directed dependency graph constructed from CycloneDX JSON SBOMs and rendered using Cytoscape.js. The visualisation supports topology exploration, node inspection, and interactive editing of the dependency structure, providing the foundational data model upon which all subsequent analysis is performed. (Chapter 4)
2. **Multi-Source Vulnerability Analysis.** We implement a vulnerability integration pipeline that queries NVD, OSV, and GitHub Security Advisories, deduplicates findings across sources, and computes full CVSS v3.1 base scores. Vulnerable components are overlaid on the dependency graph and visually distinguished by severity band, giving the analyst an immediate picture of which nodes in the graph carry known vulnerabilities. (Chapter 5)
3. **Graph-Based Structural Risk Analysis.** We apply a suite of graph-theoretic algorithms to the dependency graph to support analyst decision-making beyond raw CVE listings. Impact traversal identifies all components that depend — directly or transitively — on a given vulnerable node, making the blast radius of any vulnerability immediately visible. Centrality analysis and articulation point detection identify the components whose structural position makes them highest-priority targets for monitoring, regardless of whether they currently carry a known CVE. Community detection reveals the modular structure of the dependency ecosystem, supporting higher-level reasoning about isolation and blast containment. (??)
4. **Empirical Evaluation.** We evaluate the framework against a dataset of real-world open-source project SBOMs, collected at scale from public repositories. Through graph-level and node-level metrics we demonstrate that the approach surfaces structural risk information that is not available through flat-inventory analysis, and we discuss the implications for security analyst workflows. (??)

1.5 Scope and Limitations

This section defines the explicit boundaries of the work to prevent misreading of the claims made in subsequent chapters.

SBOM format. SBOM Lens supports CycloneDX JSON as its sole input format. SBOMs in SPDX or SWID formats are not supported. This is a deliberate design boundary rather than an oversight: CycloneDX JSON provides the richest representation of component dependency relationships within the OWASP ecosystem, and its structured dependencies array maps directly onto a directed graph without the need for format translation. Forward-compatibility with other formats is left as future work.

Evaluation methodology. The evaluation presented in ?? is metric-based: it reports graph-level and node-level measures computed on real-world open-source project SBOMs. The thesis does not include a formal usability study, a user experiment, or any assessment of whether security analysts find the tool’s output actionable in practice. Such a study would require ethical approval and a participant cohort that falls outside the scope of this work.

Features out of scope. Three capabilities that naturally extend the SBOM Lens concept are explicitly excluded from this thesis. *SBOM comparison and delta analysis* — the comparison of two SBOMs for the same project across time to detect dependency changes — is a distinct analytical problem that would require a temporal data model not present in the current system. The *Topology Optimiser* — automated recommendations for restructuring the dependency graph to reduce structural risk — requires a constraint satisfaction or optimisation layer beyond the scope of this work. Finally, SBOM Lens operates on source-level SBOMs generated from package manifests; SBOMs derived from compiled binaries or container images raise format and toolchain questions that are not addressed here.

Novelty claims. This thesis does not claim that the graph-theoretic algorithms applied are themselves novel — articulation point detection, centrality analysis, and community detection are well-established techniques. The contribution lies in their systematic application to SBOM dependency graphs as a coherent risk assessment framework, and in the empirical demonstration of what they reveal about real-world software supply chains.

1.6 Thesis Structure

The remainder of this dissertation is organised as follows.

Chapter 2 establishes the theoretical and technical foundations required to understand SBOM Lens. It covers the concept of a Software Bill of Materials and the CycloneDX format, the software supply chain security landscape, the fundamentals of graph theory and network science, and the vulnerability databases and scoring systems — NVD, OSV, and CVSS v3.1 — that underpin the vulnerability analysis.

Chapter 3 surveys the existing landscape of SBOM analysis tools — including Dependency-Track, grype, syft, and Sunshine/CycloneDX — alongside graph-based security research and supply chain attack case studies. It identifies the gap that SBOM Lens addresses: the absence of graph-structural risk analysis in current tooling.

Chapter 4 describes the architecture of SBOM Lens. It presents the React and FastAPI

system design, the CycloneDX JSON parsing pipeline, the construction of the directed dependency graph, and the interactive SBOM Graph visualisation built with Cytoscape.js.

Chapter 5 presents the multi-source vulnerability integration pipeline, covering NVD, OSV, and GitHub Security Advisories. It details the CVSS v3.1 base score computation model, the vulnerability management workflow, and the graph overlay that visually surfaces vulnerable nodes by severity band.

?? presents the structural risk assessment algorithms — articulation point detection, Shannon entropy of the degree distribution, diamond dependency detection, and k-core decomposition — and the graph analysis methods: Louvain community detection, the HITS algorithm, and eigenvector centrality. For each technique, the chapter explains the theoretical basis and its interpretation in the context of software supply chain risk.

?? evaluates the framework against a set of real-world open-source project SBOMs. It reports graph-level and node-level metrics, vulnerability findings, and structural analysis results, and discusses what the analysis reveals about structural risk in software supply chains.

?? summarises the contributions of this work, maps them back to the research objectives stated in this chapter, acknowledges the limitations of the current implementation, and proposes directions for future research.

2

Background

Modern software systems depend on hundreds or thousands of third-party components whose internals their developers never inspect. Understanding, securing, and analysing these dependency networks requires a common vocabulary spanning three fields that do not normally overlap in software engineering curricula: supply chain security, graph theory, and vulnerability scoring. This chapter establishes that vocabulary. Every concept defined here reappears by name in [Chapter 4](#), [Chapter 5](#), and ?? without re-definition. The chapter is organised in dependency order: [Section 2.1](#) defines the data format that SBOM Lens ingests; [Section 2.2](#) frames the threat landscape that motivates the analysis; [Section 2.3](#) and [Section 2.4](#) establish the structural analysis concepts applied to that data; and [Section 2.5](#) defines the scoring and database standards used to annotate the graph.

2.1 Software Bill of Materials and CycloneDX

A Software Bill of Materials is the data foundation of SBOM Lens. This section defines what an SBOM is, surveys the formats in which SBOMs are encoded, and specifies exactly which fields of the CycloneDX format SBOM Lens consumes.

2.1.1 What is a Software Bill of Materials?

A Software Bill of Materials (SBOM) is a machine-readable inventory of all software components, their versions, licences, and dependency relationships present in a given software artefact. The analogy to the ingredient label on food packaging is precise: just as a food label provides a standardised, legally mandated declaration of what a product contains, an SBOM provides a standardised, machine-processable declaration of what a software product is made of ([NTIA2021](#)).

The US National Telecommunications and Information Administration (NTIA) defines seven minimum data fields that every conformant SBOM must contain: Supplier Name, Component Name, Version of the Component, Other Unique Identifiers

(such as a Package URL, CPE, or SWID), Dependency Relationship, Author of SBOM Data, and Timestamp (**NTIA2021**). These fields define the baseline for interoperability across tooling and databases. Of the seven, the Dependency Relationship field is the most consequential for SBOM Lens: it is the only field that encodes the graph structure of the software system rather than the properties of individual components.

The regulatory impetus for SBOM adoption has grown substantially since 2021. US Executive Order 14028 (May 2021) mandated SBOMs for software sold to the US federal government; EO 14144 (January 2025) extended and reinforced these requirements (**EO14028**). The European Cyber Resilience Act (CRA), which entered into force in December 2024, similarly requires software component transparency for products with digital elements sold in the EU market (**CRA2024**). SBOMs have therefore moved from a voluntary best practice to a regulatory requirement in the two largest software markets in the world.

For SBOM Lens, the SBOM is the sole input artefact. Without the explicit dependency relationship field encoded in a machine-readable format, the graph construction pipeline described in **Chapter 4** would not be possible. The choice of CycloneDX as the supported format — examined in **Section 2.1.3** — is motivated directly by the richness and explicitness of its dependency graph representation.

2.1.2 SBOM Format Landscape

Three machine-readable SBOM formats exist in practice: CycloneDX (OWASP, originally 2017), SPDX (Linux Foundation, originally 2010, now ISO/IEC 5962), and SWID Tags (ISO/IEC 19770-2). Each was designed with a different primary use case, and that design intent has significant practical consequences for graph-based analysis.

SPDX was created primarily for licence compliance. Its relationship model is broad — it defines approximately twenty-five relationship types including DESCRIBES, CONTAINS, DEPENDS_ON, GENERATED_FROM, and OPTIONAL_DEPENDENCY_OF — but the dependency relationship is represented implicitly as one type among many, and SPDX 2.3 has no native structure for embedding vulnerability data. SPDX is widely adopted for licence auditing workflows but does not natively support the graph structure required for the analysis SBOM Lens performs.

CycloneDX was designed from the outset for security use cases: vulnerability tracking, dependency analysis, and supply chain integrity verification. Its key structural advantage over SPDX is the explicit separation of components and their dependency relationships into two distinct top-level arrays. A CycloneDX document's `dependencies[]` array makes the dependency graph unambiguous: each entry declares a component and its set of direct dependents, with no inference required. CycloneDX 1.6 added native support for VEX (Vulnerability Exploitability eXchange) documents via a `vulnerabilities[]` array. The current version, CycloneDX 1.7, was published in 2025 and is standardised as ECMA-424 (**OWASPCycloneDX**).

SWID Tags are excluded from consideration for SBOM Lens because they lack a

standardised dependency relationship field; they identify installed software but do not encode the graph structure between components.

SBOM Lens supports CycloneDX JSON exclusively. This is an explicit design boundary, not an oversight: CycloneDX provides the most explicit and semantically unambiguous graph representation available in any SBOM format, eliminating the inference step that SPDX dependency reconstruction would require.

2.1.3 CycloneDX JSON Structure and SBOM Lens Fields

A CycloneDX JSON document has a rich schema, but SBOM Lens consumes three top-level constructs. The `metadata.component` object describes the root of the SBOM — the project being analysed, with its name, version, and type. The `components[]` array contains one entry per dependency component, each carrying at minimum a `bom-ref` (a stable local document identifier), a name, a version, a `purl` (Package URL), and a type. The `dependencies[]` array encodes the graph structure: each entry takes the form `{"ref": "bom-ref-A", "dependsOn": ["bom-ref-B", "bom-ref-C"]}`, declaring that the component identified by `bom-ref-A` directly depends on both `bom-ref-B` and `bom-ref-C` (OWASPCycloneDX).

The `bom-ref` field is SBOM Lens's graph node identifier. It is a stable, local reference that uniquely identifies a component within a single CycloneDX document. When `bom-ref` is absent — which occurs in some generator tools — SBOM Lens falls back to the `purl` field, and further to the `name` field as a last resort. This three-step fallback chain is implemented in the graph construction pipeline described in [Chapter 4](#).

The directed dependency graph is constructed directly from these fields. The vertex set V comprises all entries in `components[]` plus the root component from `metadata.component`. The edge set E is derived from `dependencies[]`: for each entry with `ref = u` and `dependsOn = [v1, v2, ...]`, SBOM Lens adds directed edges $u \rightarrow v_1$, $u \rightarrow v_2$, ... A directed edge from component u to component v means “ u depends on v .”

The `purl` field serves a second purpose beyond node identification. A Package URL encodes the package ecosystem, namespace, name, and version in a single string — for example, `pkg:npm/lodash@4.17.21` or `pkg:pypi/requests@2.31.0` — enabling cross-database vulnerability matching. OSV uses `purl` natively; NVD uses CPE but can be cross-referenced via `purl`. The `purl`-to-database matching pipeline is described in [Chapter 5](#). The `purl` specification is standardised as ECMA-427 ([PurlSpec](#)).

Fields not consumed by SBOM Lens in the current implementation include the `vulnerabilities[]` array (VEX documents), `services[]`, `compositions[]`, and `externalReferences[]`. Vulnerability data is sourced directly from NVD, OSV, and GitHub Security Advisories rather than from embedded VEX data; this design decision and its trade-offs are discussed in ??.

2.2 Software Supply Chain Security

Before describing the algorithms SBOM Lens applies, it is necessary to establish what the analysis is defending against. This section defines the software supply chain as a concept, characterises the threat categories it faces, and summarises the regulatory responses that have raised SBOM analysis from an optional practice to a compliance requirement.

2.2.1 The Software Supply Chain

The software supply chain encompasses all the components, libraries, tools, and processes — and the people and organisations behind them — involved in developing, building, packaging, and distributing a software product. The supply chain metaphor is precise: each upstream dependency is a supplier whose code is inherited by all downstream consumers. Vulnerabilities, backdoors, and deprecated components in any supplier propagate transitively to every consumer in the dependency graph.

The scale of this dependency is larger than most practitioners appreciate. The Synopsys Open Source Security and Risk Analysis (OSSRA) 2024 report found that 96% of commercial codebases contain open-source components (**OSSRA2024**). More consequentially, direct dependencies — those explicitly declared in a project’s manifest — represent only a fraction of the full dependency tree. Transitive dependencies, the dependencies of dependencies recursively, routinely outnumber direct dependencies by an order of magnitude in large projects.

The log4shell vulnerability (CVE-2021-44228, December 2021) illustrates the practical consequence. Apache Log4j was a transitive dependency in millions of Java applications; many of the affected development teams were entirely unaware that their software included it. A flat SBOM inventory can reveal whether Log4j is present; it cannot reveal how deeply embedded it is, how many other components depend on it, or how a vulnerability in it would propagate through the dependency graph. This structural blindspot is the gap that SBOM Lens addresses through the graph analysis methods described in ??.

2.2.2 Attack Taxonomy

Software supply chain attacks exploit the transitive trust that consumers place in upstream suppliers. Ladisa et al. (2023) provide a systematic taxonomy of open-source supply chain attacks in a peer-reviewed Systematisation of Knowledge (SoK) paper (**Ladisa2023**). Four primary categories are relevant to the SBOM analysis context.

Build pipeline compromise describes attacks in which an adversary infiltrates a vendor’s CI/CD system or release process and injects malicious code into an otherwise legitimate software artefact. The SolarWinds SUNBURST attack (December 2020, CISA Alert AA20-352A) is the canonical example: malicious code was inserted into a signed software update and distributed to approximately 18,000 organisations, including US

government agencies, via a trusted vendor relationship. The resulting component appears legitimate in any SBOM because it was produced by the authentic vendor.

Compromised maintainer attacks involve an adversary gaining legitimate commit access to an open-source project — typically through social engineering over months or years — and then inserting a backdoor. The xz-utils backdoor (CVE-2024-3094, CVSS 10.0, March 2024) is a recent and technically sophisticated example: a two-year campaign resulted in a backdoor narrowly avoiding widespread compromise of OpenSSH on systemd-based Linux distributions.

Dependency confusion exploits the resolution logic of package managers: an attacker publishes a malicious package on a public registry with the same name as a private internal package, causing the package manager to resolve the public (malicious) version instead of the private (legitimate) one.

Typosquatting relies on registering package names visually similar to widely-used packages — for example, `requests-oauthlib2` alongside `requests-oauthlib` — to capture installations triggered by typographical errors.

Graph-based SBOM analysis complements detection for the latter two categories. K-core decomposition (??) reveals how deeply embedded a potentially substituted package is in the dependency network, providing a structural triage signal for incident response.

2.2.3 Policy and Standards Response

The regulatory and standards response to supply chain attacks has accelerated since 2021. US Executive Order 14028 (May 2021) mandated SBOMs for software sold to the US federal government, with Section 10(j) providing the authoritative regulatory definition of SBOM (EO14028). EO 14144 (January 2025) reinforced and extended these requirements. The European Cyber Resilience Act, which entered into force in December 2024, requires manufacturers and developers of products with digital elements sold in the EU to document software components and address known vulnerabilities throughout the product lifecycle (CRA2024).

At the standards level, the NIST Secure Software Development Framework (SP 800-218) describes secure development practices at the process level (NIST_SSDF). Supply-chain Levels for Software Artifacts (SLSA) provides a complementary four-level maturity framework for source integrity, build integrity, and provenance attestation, operationalising trust verification in CI/CD pipelines (SLSA). CISA has further published guidance on SBOM minimum elements, sharing mechanisms, and tooling recommendations. These frameworks collectively define the regulatory and operational context in which SBOM Lens operates.

2.3 Graph Theory Fundamentals

The dependency structure encoded in a CycloneDX SBOM is a directed graph. This section establishes the graph-theoretic vocabulary used throughout the thesis. Definitions are given mathematically, with an SBOM interpretation for each concept, and a forward-reference to the chapter where each is applied in SBOM Lens. The primary reference is [CLRS4th](#).

2.3.1 Directed Graphs and Basic Definitions

A *directed graph* (digraph) $G = (V, E)$ consists of a vertex set V and a set of directed edges $E \subseteq V \times V$ ([CLRS4th](#)). For SBOM Lens, V is the set of software components present in the SBOM and E represents dependency relationships: a directed edge $(u, v) \in E$ means that component u directly depends on component v .

The *in-degree* of a vertex v , written $\deg^-(v)$, is the number of directed edges incoming to v — equivalently, the number of components that directly depend on v . The *out-degree* $\deg^+(v)$ is the number of directed edges outgoing from v — the number of components on which v directly depends. In an SBOM dependency graph, a component with high in-degree is one that many others depend upon; a component with high out-degree has many direct dependencies of its own. Both are indicators of structural importance examined in [??](#).

The *adjacency matrix* $\mathbf{A}$ of G is an $|V| \times |V|$ matrix where entry $A_{ij} = 1$ if the directed edge $i \rightarrow j$ exists and $A_{ij} = 0$ otherwise. Real SBOM graphs are typically sparse — a graph with thousands of components may have only tens of thousands of edges — so $\mathbf{A}$ is stored in compressed sparse format in practice. The adjacency matrix is the basis for eigenvector centrality analysis described in [Section 2.4.3](#).

A *root node* is a vertex with in-degree zero: no other component in the SBOM depends on it. In SBOM Lens, the root node is the project being analysed, identified by `metadata.component` in the CycloneDX document. The *depth* of a component is the length of the shortest directed path from the root to that component; depth-1 components are direct dependencies, depth- k components are k -hop transitive dependencies.

2.3.2 Directed Acyclic Graphs and Cycles

A *directed acyclic graph* (DAG) is a directed graph that contains no directed cycle ([CLRS4th](#)). Dependency graphs are ideally DAGs: if package A depends on package B which depends on A , neither can be resolved without the other, and standard package managers report this as an error. A DAG admits a *topological ordering* — a linear arrangement of all vertices such that every edge goes from earlier to later — which corresponds to a valid build or installation order.

In practice, SBOMs generated by real tools may contain directed cycles, arising from circular version-pinned dependencies or artefacts of SBOM generation tooling. SBOM Lens models the dependency graph as a general digraph (not an enforced DAG) using

NetworkX's `DiGraph` class. Cycles are identified by computing *strongly connected components* (SCCs): a set of vertices in which every vertex is reachable from every other via directed paths. An SCC of size greater than one indicates a directed cycle. SBOM Lens flags such SCCs as a structural anomaly, applying Tarjan's algorithm in $O(|V| + |E|)$ time (CLRS4th). This analysis is described in ??.

A *diamond dependency* occurs when two distinct dependency paths from a common ancestor converge on the same descendant: component A depends on both B and C , and both B and C depend on D . If B requires $D @1.0$ and C requires $D @2.0$, a version conflict exists. Diamond dependencies are a structural risk indicator detected by SBOM Lens and reported as part of the structural risk assessment in ??.

2.3.3 Connectivity and Reachability

A directed graph is *weakly connected* if the underlying undirected graph — obtained by ignoring edge directions — is connected. It is *strongly connected* if a directed path exists between every ordered pair of vertices. Most SBOM dependency graphs are weakly connected (the components form a single ecosystem) but not strongly connected, because dependency direction is asymmetric: a library does not depend on the applications that use it (CLRS4th).

The *ancestor set* of a vertex v , written $\text{anc}(v)$, comprises all vertices from which v is reachable via directed paths — all components that transitively depend on v . In vulnerability analysis, $|\text{anc}(v)|$ is the *blast radius* of a vulnerability in v : the number of components that would be affected if v were compromised. Conversely, the *descendant set* $\text{desc}(v)$ comprises all vertices reachable from v — all transitive dependencies of v . SBOM Lens computes these sets using NetworkX's `nx.ancestors` and `nx.descendants` functions. Both are applied in Chapter 5 as a vulnerability impact quantification method.

Reachability is computed via breadth-first search (BFS) and depth-first search (DFS), both running in $O(|V| + |E|)$ (CLRS4th). BFS from the root visits components in non-decreasing depth order, establishing the dependency layering of the graph. DFS is used for cycle detection and for the articulation point algorithm described in the following subsection.

2.3.4 Articulation Points and Bridge Edges

An *articulation point* (also called a cut vertex) in a connected undirected graph is a vertex whose removal — along with its incident edges — increases the number of connected components (CLRS4th). In the context of a software dependency graph, an articulation point is a component whose removal would fragment the dependency network into disconnected parts. Such a component is a *single point of failure*: a vulnerability, deprecation, or licence change affecting it propagates to every component in the severed partition, with no alternative dependency path.

Detection is performed by a DFS-based algorithm running in $O(|V|+|E|)$ time (CLRS4th). For each vertex v , the algorithm tracks the discovery time $\text{disc}[v]$ (the DFS visit order) and the lowest discovery time reachable from the subtree rooted at v via back edges, $\text{low}[v]$. A vertex v is an articulation point if any DFS-tree child w of v satisfies $\text{low}[w] \geq \text{disc}[v]$, meaning no back edge in w 's subtree can bypass v to reach v 's ancestors. Because SBOM dependency graphs are directed, SBOM Lens applies the algorithm to the undirected projection of the graph: `nx.articulation_points(G.to_undirected())`. The set of articulation points is a primary output of the structural risk assessment in ??.

A *bridge* (or cut edge) is an edge whose removal disconnects the graph — the edge-level analogue of an articulation point. SBOM Lens identifies bridge edges as structural “firebreaks”: dependency relationships whose severance partitions the ecosystem. Bridge edges are reported alongside articulation points in ??.

2.4 Network Science Concepts

Graph theory provides the foundational vocabulary for describing individual graph structures. Network science extends this vocabulary to characterise the global statistical properties of large graphs — properties that emerge from the aggregate behaviour of thousands of nodes and edges and that cannot be read from any single vertex or edge in isolation. This section introduces the network science methods applied in ??, with mathematical definitions, supply chain interpretations, and forward references. Primary references are Barabasi2016 and Newman2018.

2.4.1 Scale-Free Networks and Power-Law Degree Distributions

Many real-world networks — the World Wide Web, citation networks, biological protein-interaction networks — exhibit degree distributions that follow a power law: $P(k) \sim k^{-\gamma}$ where $\gamma \in [2, 3]$ for most empirical cases (Barabasi1999). Unlike the Poisson distribution characteristic of random Erdős–Rényi graphs, a power-law distribution has no characteristic scale: a small number of nodes (“hubs”) have very high degree while the vast majority have low degree. Networks with power-law degree distributions are called *scale-free networks*.

Barabási and Albert (1999) demonstrated that scale-free networks emerge naturally from two mechanisms: network growth (nodes are added over time) and preferential attachment (new nodes connect to existing nodes with probability proportional to their current degree) (Barabasi1999). This “rich get richer” dynamic explains hub emergence. Open-source package ecosystems exhibit both mechanisms: new packages are added continuously, and new packages tend to depend on already-popular libraries.

If the degree distribution of a software dependency graph follows a power law, a small number of packages serve as hubs on which a very large number of others depend. A vulnerability in a hub component has a blast radius far exceeding that of

a vulnerability in a leaf node. The degree distributions of the SBOMs evaluated in ?? are reported empirically; hub components are identified and reported by SBOM Lens as part of the structural risk assessment in ??.

2.4.2 Small-World Networks

Watts and Strogatz (1998) characterised a class of networks that simultaneously exhibit high clustering coefficient C — indicating tightly knit local neighbourhoods — and short average shortest path length L comparable to a random graph of the same size (**WattsStrogatz1998**). The small-world coefficient $\sigma = (C/C_{\text{rand}})/(L/L_{\text{rand}})$ quantifies this property: $\sigma \gg 1$ indicates small-world structure.

In software dependency graphs, high clustering can emerge from ecosystem sub-communities: all React-ecosystem packages may share mutual dependencies on a common set of utilities, for example, creating dense local neighbourhoods. Short average path lengths mean that a vulnerability can propagate across the entire graph in relatively few hops, even in a large SBOM.

The clustering coefficient and average path length of analysed SBOMs are reported as structural metrics in ??, providing empirical evidence for or against small-world structure in real dependency graphs.

2.4.3 Centrality Measures

Centrality measures quantify the structural importance of individual nodes in a graph, each from a different perspective on what “importance” means.

Degree centrality is the simplest: a node’s degree (or normalised degree) counts its direct connections. In the directed SBOM graph, normalised in-degree centrality identifies components that many others depend on directly; normalised out-degree centrality identifies components with many direct dependencies. Both are computable in $O(|V| + |E|)$ (**Barabasi2016**).

Betweenness centrality measures the fraction of shortest paths between all pairs of vertices that pass through a given vertex v :

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from s to t and $\sigma_{st}(v)$ is the number of those paths passing through v . High betweenness identifies relay nodes — bottlenecks through which dependency resolution paths are routed. The Brandes algorithm computes betweenness centrality in $O(|V| \cdot |E|)$ (**Barabasi2016**).

The *HITS algorithm* (Hyperlink-Induced Topic Search, Kleinberg 1999) defines two mutually reinforcing per-node scores (**Kleinberg1999**). The *hub score* of a node is proportional to the sum of the authority scores of the nodes it points to; the *authority score* is proportional to the sum of the hub scores of nodes pointing to it. In an SBOM

dependency graph, high-authority components are fundamental libraries (widely depended upon by hub components); high-hub components are aggregators that depend on many authoritative libraries. HITS scores are computed iteratively until convergence and applied in ??.

Eigenvector centrality assigns each node a score proportional to the sum of its neighbours' scores, solved as the leading eigenvector of the adjacency matrix $\mathbf{A}$. It accounts for the quality as well as the quantity of a node's connections: being depended upon by important components counts more than being depended upon by peripheral ones.

2.4.4 Community Detection and Modularity

A *community* (or module) is a subgraph that is more densely connected internally than it is to the rest of the graph (Barabasi2016). In software dependency graphs, communities correspond naturally to ecosystem sub-communities: all packages from the same framework, or all packages serving the same functional role, tend to depend on each other more than on packages from a different ecosystem.

Community quality is quantified by *modularity* Q :

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where $m = |E|$, k_i is the degree of node i , and $\delta(c_i, c_j) = 1$ if and only if nodes i and j belong to the same community. Modularity compares the density of intra-community edges against the expectation under a random null model with the same degree sequence; values above approximately 0.3 are generally considered to indicate meaningful community structure (Barabasi2016).

The *Louvain algorithm* (Blondel et al., 2008) is a two-phase greedy modularity optimisation (Blondel2008). In the first phase, each node is assigned to the community of one of its neighbours that maximises the local modularity gain. In the second phase, all nodes in a community are aggregated into a single super-node, and the process repeats on the reduced graph. The algorithm terminates when no further modularity gain is possible. Its practical complexity is $O(n \log n)$, making it tractable on large SBOM graphs. SBOM Lens applies the Louvain algorithm to the undirected projection of the dependency graph and reports the resulting community structure in ??.

From a security perspective, detected communities reveal which components form tightly coupled subsystems. A vulnerability that enters one high-modularity community tends to spread within it, contained by the sparse inter-community edges. Communities with many inter-community connections are higher-risk propagation bridges.

2.4.5 K-Core Decomposition

The *k-core* of a graph is the maximal induced subgraph in which every vertex has degree at least k within that subgraph (Batagelj2003). The *coreness* (or core number) of

a vertex v is the largest k such that v belongs to the k -core. Coreness is a measure of how deeply embedded a component is in the dependency network.

K-core decomposition is computed by the shell-peeling algorithm of Batagelj and Zaversnik (2003), which runs in $O(|E|)$ time (Batagelj2003). The algorithm iteratively removes all vertices with degree less than 1 (the 0-shell), then all vertices with degree less than 2 (the 1-shell), and so on, until no vertices remain. The innermost core — the subgraph with maximum coreness $k_{\max}$ — is the architectural backbone of the dependency network: components that are mutually highly interconnected and collectively difficult to remove.

In the supply chain context, high-coreness components are the most entrenched in the dependency network. They are depended upon by many others (high in-degree) and themselves depend on many others (high out-degree). A vulnerability in a high-core component has maximum blast radius and maximum replacement difficulty: replacing it requires finding an alternative that satisfies all components in the inner core simultaneously. SBOM Lens computes coreness using `nx.core_number(G)` and reports the maximum coreness and the composition of the innermost core. This analysis is applied in ??.

2.4.6 Propagation Modelling --- Future Directions

Graph-based vulnerability analysis can be extended beyond the structural metrics described above to include propagation modelling — explicitly simulating how a compromise spreads through the dependency network over time. Epidemic models borrowed from epidemiology, such as the SIR (Susceptible–Infected–Recovered) framework (PastorSatorras2001), have been applied to software dependency graphs to estimate the fraction of components that would be affected by a single-component compromise, using the graph’s spectral radius as the key parameter governing spread. Similarly, graph-level structural fingerprinting via Weisfeiler–Lehman hashing can enable change detection between SBOM versions without full graph comparison.

These techniques represent a natural extension of the SBOM Lens analysis pipeline but are outside the scope of the current work. SBOM Lens focuses on static structural risk assessment — identifying high-risk components from the graph topology and vulnerability overlay — rather than dynamic propagation simulation. Propagation modelling is identified as a direction for future work in ??.

2.5 Vulnerability Databases

SBOM Lens overlays vulnerability data on the dependency graph to identify which components carry known security risks and to quantify the severity of those risks. This section defines the three databases from which SBOM Lens sources vulnerability data — CVE/NVD, OSV, and GitHub Security Advisories — and establishes the CVSS v3.1 scoring system used to quantify severity. Definitions here are referenced without re-

explanation in [Chapter 5](#).

2.5.1 CVE and the National Vulnerability Database

Common Vulnerabilities and Exposures (CVE) is a programme maintained by MITRE (funded by CISA and the US government) that assigns unique identifiers to publicly disclosed software vulnerabilities (**MITRE_CVE**). A CVE ID takes the form `CVE-<YEAR>-<SEQUENCE>`, for example `CVE-2021-44228` for the `log4shell` vulnerability. Each CVE entry provides a standardised description, a list of affected products, references to public advisories and patches, and severity scores. CVE is the canonical cross-database identifier: entries in NVD, OSV, and GitHub Security Advisories all reference CVE IDs when applicable, enabling deduplication across sources.

The NIST National Vulnerability Database (NVD) enriches CVE entries with CVSS base scores (versions 2.0, 3.x, and 4.0), Common Platform Enumeration (CPE) product matching strings, and categorical severity ratings (**NVD**). NVD provides the authoritative CVSS v3.1 base scores for most published CVEs and is the primary reference for severity data in the security industry.

SBOM Lens queries NVD via its REST API v2.0 (endpoint `/rest/json/cves/2.0` for CVE lookup). NVD is used as a secondary source after OSV because NVD's primary product identifier is CPE rather than purl, requiring a name-and-version mapping step that introduces matching imprecision. The CPE format, `cpe:2.3:a:<vendor>:<product>:<version>:...`, is less precise for open-source packages than purl-based matching: the same package may have multiple CPE strings or none at all. This limitation is discussed in [Chapter 5](#).

2.5.2 Open Source Vulnerabilities (OSV)

The Open Source Vulnerabilities (OSV) database is a distributed vulnerability database designed natively for open-source ecosystems (**OSV**). Its schema, maintained by the OpenSSF, defines version range bounds using `introduced` and `fixed` fields rather than fixed version lists, enabling precise matching against any version in a given range. OSV aggregates vulnerability data from over fifty upstream sources including RustSec, PyPI advisories, npm advisories, the GitHub Advisory Database, the Debian Security Tracker, and the Alpine Security Database.

Each OSV entry carries an `aliases` field that cross-references CVE IDs and GHSA identifiers, enabling deduplication across sources. OSV's query API (`/v1/query`) accepts a package purl directly, making it the most precise source for open-source component vulnerability matching and the primary vulnerability source in SBOM Lens. The integration pipeline is described in [Chapter 5](#).

2.5.3 GitHub Security Advisories

The GitHub Advisory Database is a curated collection of security advisories for open-source packages, using GHSA identifiers in the format `GHSA-xxxx-xxxx-xxxx` (**GHSA**). Its strongest ecosystem coverage is for npm (JavaScript), PyPI (Python), RubyGems, Maven, NuGet (.NET), Go, and Rust. GitHub advisories are published in OSV format and are therefore aggregated by the OSV database.

Because GHSA advisories propagate to OSV, querying OSV subsumes most GHSA coverage. SBOM Lens additionally queries GHSA directly via GitHub’s GraphQL API for npm and PyPI packages to capture advisories published on GitHub before they have propagated to the OSV aggregation pipeline, exploiting the recency advantage for those two ecosystems.

The three-source integration — NVD, OSV, and GHSA — is a deliberate design choice to maximise vulnerability coverage across package ecosystems and version ranges. Findings from all three sources are deduplicated using CVE ID as the primary key; where the same CVE appears in multiple sources, the highest severity score is retained. The deduplication logic and its trade-offs are presented in full in [Chapter 5](#).

2.5.4 CVSS v3.1 Scoring System

The Common Vulnerability Scoring System (CVSS) v3.1 is the standard for quantifying the severity of software vulnerabilities (**CVSS31**). The specification is maintained by FIRST.org (Forum of Incident Response and Security Teams). CVSS produces a numeric base score on the interval $[0.0, 10.0]$ from a set of standardised metrics that characterise the intrinsic properties of the vulnerability, assessed at the time of publication.

The base metric group comprises eight metrics divided into two sub-groups plus a Scope modifier. The four *exploitability metrics* characterise how the vulnerability can be accessed: **Attack Vector** (AV: Network, Adjacent, Local, or Physical), **Attack Complexity** (AC: Low or High), **Privileges Required** (PR: None, Low, or High), and **User Interaction** (UI: None or Required). The **Scope** modifier (S: Unchanged or Changed) indicates whether a successful exploit can affect components beyond the vulnerable component — for example, when a container escape vulnerability affects the host operating system. The three *impact metrics* characterise the effect on the CIA triad of the affected system: **Confidentiality Impact** (C), **Integrity Impact** (I), and **Availability Impact** (A), each rated None, Low, or High (**CVSS31**).

The base score is computed from these eight metrics via the following formula (**CVSS31**). Let $ISS = 1 - [(1 - C)(1 - I)(1 - A)]$ where C , I , and A are numerical values from the specification’s lookup table. The impact sub-score is:

$$\text{Impact} = \begin{cases} 6.42 \times ISS & \text{if Scope Unchanged} \\ 7.52 \times (ISS - 0.029) - 3.25 \times (ISS - 0.02)^{15} & \text{if Scope Changed} \end{cases}$$

The exploitability sub-score is $\text{Exploitability} = 8.22 \times \text{AV} \times \text{AC} \times \text{PR} \times \text{UI}$, where each metric takes its numerical value from the lookup table. The base score is then $\text{BaseScore} = \min(\text{Impact} + \text{Exploitability}, 10)$, rounded up to the nearest 0.1. If $\text{Impact} = 0$, the base score is 0.0 regardless of exploitability.

FIRST.org defines five severity bands: None (0.0), Low (0.1–3.9), Medium (4.0–6.9), High (7.0–8.9), and Critical (9.0–10.0). The log4shell vulnerability (CVE-2021-44228) illustrates the maximum score: its vector string

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H yields $\text{BaseScore} = 10.0$ because the vulnerability is network-exploitable without authentication, affects scope beyond the vulnerable component, and has maximum confidentiality, integrity, and availability impact. SBOM Lens uses these severity bands to colour-code vulnerable nodes on the SBOM graph — red for Critical and High, orange for Medium, yellow for Low — as described in [Chapter 5](#).

The Exploit Prediction Scoring System (EPSS, also maintained by FIRST.org) complements CVSS by estimating the probability that a CVE will be exploited in the wild within thirty days, using a machine-learning model trained on observed exploitation data (**EPSS**). While CVSS measures the potential severity of a vulnerability if exploited, EPSS measures the likelihood of exploitation. SBOM Lens does not currently integrate EPSS scores; this is noted as a limitation and a direction for future work in ??.

3

Related Work

Situating SBOM Lens within existing work requires surveying three bodies of literature that do not ordinarily appear side by side: the practical tooling landscape for SBOM analysis, the academic research programme on graph-based software security, and a pair of supply chain incidents that illustrate the structural blindspots the tools leave open. This chapter covers each in turn. Section 3.1 surveys four open-source SBOM tools using a feature matrix, weaving in the CVSS/EPSS scoring context established in Chapter 2. Section 3.2 organises the academic literature into four research clusters and connects each to an SBOM Lens design decision. Section 3.3 re-examines the SolarWinds SUNBURST compromise and the log4shell disclosure through the lens of SBOM graph analysis rather than incident narrative. Section 3.4 synthesises the survey into four named research gaps that SBOM Lens addresses in Chapter 4, Chapter 5, and ??.

3.1 Existing SBOM Analysis Tools

The SBOM tooling ecosystem has grown rapidly since the US Executive Order 14028 of 2021 accelerated adoption across government-adjacent supply chains. Four open-source tools are in widespread production use: Dependency-Track (OWASP), grype and syft (Anchore), and Sunshine (CycloneDX project). Commercial tools such as FOSSA also exist but are beyond the scope of this survey, which focuses on open-source solutions whose internals can be examined and extended. Closest to the graph-adjacent end of the spectrum is GUAC (Graph for Understanding Artifact Composition), which joined OpenSSF as an incubating project in March 2024 (**GUAC2024**); GUAC constructs graphs across multiple SBOMs at portfolio scale, which is a complementary but distinct scope from the within-SBOM structural analysis that SBOM Lens addresses.

Table 3.1 compares the four open-source tools along four dimensions: SBOM format support, graph capability, vulnerability integration, and scope/purpose. SBOM Lens appears as the final row, separated by a rule, so the capability gaps are structurally visible.

Table 3.1: Comparison of SBOM analysis tools across four capability dimensions. SBOM Lens (last row) is the reference implementation evaluated in this thesis.

Tool	Format Support	Graph Capability	Vuln Integration	Scope
Dependency-Track	CycloneDX (preferred), SPDX	None — flat component list	NVD, OSV, GHSA, EPSS, Snyc, VulnDB	Portfolio-wide continuous monitoring
grype	CycloneDX, SPDX	None — flat CVE list	Aggregated DB (NVD, OSV, GHSA), EPSS, CISA KEV	Point-in-time vulnerability scan; CI/CD
syft	CycloneDX (JSON+XML), SPDX	None — SBOM generator only	None (generation only)	SBOM generation from images, filesystems, source
Sunshine	CycloneDX JSON only	Visual/hierarchical only (sunburst chart) — no graph algorithms	EPSS, CISA KEV enrichment; SQL query interface	Interactive SBOM viewer and SQL querying
SBOM Lens	CycloneDX JSON	Directed dependency graph; articulation points, k-core decomposition, HITS, Louvain community detection, degree/betweenness/closeness centrality	NVD, OSV, GHSA; CVSS v3.1 base score; vulnerability-graph overlay	Within-SBOM structural risk assessment

3.2 Graph-Based Security Research

A substantial body of academic work applies graph theory and network science to software dependency networks, primarily at the package-ecosystem scale. This section surveys four research clusters — dependency graph analysis, software vulnerability graphs, call graph and static analysis, and network science applied to software — and positions SBOM Lens relative to each. Each cluster closes with a forward reference to the SBOM Lens implementation chapter where the cluster’s insights are applied.

3.2.1 Dependency Graph Analysis

Empirical studies of package dependency networks have established that open-source ecosystems form large, scale-free graphs in which a small number of packages are transitively depended upon by the majority of the ecosystem.

3.2.2 Software Vulnerability Graphs

A second cluster of work models vulnerability propagation through dependency graphs, moving beyond per-package CVE counts toward structural propagation metrics.

3.2.3 Call Graph and Static Analysis

Call graph reachability analysis determines whether a vulnerable function is actually reachable from application code, filtering CVE noise to only genuinely exploitable paths.

3.2.4 Network Science Applied to Software

The network science literature provides the theoretical toolkit that SBOM Lens applies to individual SBOM dependency graphs.

3.3 Supply Chain Attack Case Studies

Both cases examined below were introduced in [Section 1.1](#) as evidence of the structural risks motivating SBOM Lens. Rather than re-narrating the incidents, this section frames each through the SBOM graph analysis lens: what information was encoded in the dependency graph, and what graph-theoretic metrics would have made the structural exposure visible before or immediately after disclosure.

3.3.1 SolarWinds SUNBURST (December 2020)

As introduced in [Section 1.1](#), the SolarWinds SUNBURST compromise demonstrated build-pipeline supply chain risk ([CISA_AA20_352A](#)); this subsection examines what graph-based SBOM analysis would have revealed.

3.3.2 log4shell (CVE-2021-44228, December 2021)

As introduced in [Section 1.1](#), the log4shell vulnerability exposed the hidden scale of transitive dependency risk (NIST_CVE_2021_44228); this subsection quantifies what an SBOM graph would have shown.

3.4 Gap Analysis

The survey above reveals four capabilities absent from current tooling and the research literature; SBOM Lens addresses each of these gaps. The evidence for each absence is grounded in [Table 3.1](#) and the research clusters surveyed in [Section 3.2](#).

4

System Design

Chapter 4 describes the design and implementation of SBOM Lens — the system whose absence from existing tooling [Section 3.4](#) identified as the central research gap. The chapter is organised into four sections. [Section 4.1](#) presents the three-layer architecture and the rationale behind its main design choices. [Section 4.2](#) traces the data flow from a CycloneDX JSON file upload through JSON validation, duplicate detection, component extraction, and directed-edge construction, to the in-memory `nx.DiGraph`. [Section 4.3](#) defines the node and edge data model that all algorithms in [Chapter 5](#) and ?? operate on. [Section 4.4](#) describes the interactive Cytoscape.js visualisation layer — the base rendering, layout options, and core interaction model.

4.1 System Architecture

SBOM Lens is structured as a three-layer system: a React single-page application (frontend), a FastAPI web service (backend), and a REST API that decouples them. Each layer has a single, bounded responsibility — the frontend handles rendering and user interaction, the backend holds all graph computation, and the REST API defines the contract between the two. This section presents the three layers and the design rationale that shaped their boundaries.

4.1.1 Overview and Design Rationale

SBOM Lens follows a three-layer architecture that separates user interaction, graph computation, and data transfer into distinct, independently testable concerns. The frontend (React with Cytoscape.js) is responsible for rendering the SBOM Graph, presenting the vulnerability dashboard, and mediating user input. The backend (FastAPI with NetworkX) owns all structural computation — graph construction, metric calculation, risk assessment, and vulnerability aggregation. A REST API defines the contract between the two layers, allowing either side to evolve without coupling.

The principal motivation for this separation is analytical correctness. Running graph

algorithms in the browser would require reimplementing the backend graph logic in JavaScript, duplicating code that would be harder to test and audit. By keeping computation server-side, the algorithms described in [Chapter 5](#) and ?? can be verified against known graph fixtures independently of the frontend.

Persistence is deliberately in-memory for this thesis scope. SBOMStore holds up to MAX_SBOMS SBOM records and their corresponding `nx.DiGraph` objects in process memory; the default limit is ten, configurable via environment variable. Graph state resets on server restart. This design is intentional: the system is evaluated against a fixed research dataset, and the absence of a database removes an entire class of schema-migration and query-optimisation concerns that would distract from the analytical contribution. Persistent storage is identified as future work.

The architecture also includes modules not evaluated in this thesis — a Topology Optimiser and a Comparison/Delta analysis view — which are shown in [Figure ??](#) with a visual distinction to be transparent about the system’s full scope while clearly delineating what this thesis covers. All seven registered FastAPI routers are listed for architectural completeness.

4.1.2 Frontend Layer

The frontend is a React single-page application styled with the Material-UI (MUI) component library. It exposes three primary views: the SBOM Graph (the central visualisation canvas), the Vulnerability Dashboard (aggregated CVE data and remediation status), and the Graph Analysis panel (network-science metric display). A persistent side navigation bar connects the views and hosts the SBOM selection control.

Graph rendering is delegated entirely to Cytoscape.js through a `GraphVisualizer` React component that holds the Cytoscape instance in a `useRef` hook. This architectural choice is deliberate: Cytoscape.js manages the HTML5 canvas directly and responds to user gestures with sub-millisecond latency. Routing all canvas operations through React’s virtual-DOM reconciler would introduce unnecessary re-render overhead for every pan, zoom, or node drag event. React manages application state — which SBOM is loaded, which analysis mode is active — while Cytoscape manages the visual representation.

The data layer is controlled by a compile-time constant `DATA_SOURCE` defined in the application configuration file. It switches between `'localStorage'` (an offline convenience mode used during frontend development) and `'api'` (the production path that calls the FastAPI backend). The thesis evaluation uses the `'api'` mode exclusively. The analysis toolbar, which exposes the overlay modes described in [Chapter 5](#) and ??, is visible only in API mode when an SBOM is loaded.

4.1.3 Backend Layer

The backend is a FastAPI application defined in `main.py` with seven routers registered at startup: `sboms`, `vulnerabilities`, `metrics`, `impact`, `audit`, `graph_analysis`, and

delta. Registration order is significant: the delta and graph_analysis routers are registered before sboms to prevent FastAPI's wildcard path parameter {sbom_id} from greedily intercepting fixed paths such as /sboms/compare.

The graph engine is NetworkX's DiGraph class. All structural algorithms operate directly on the stored DiGraph object: these include articulation-point detection, k-core decomposition, HITS centrality, Louvain community detection, spectral radius estimation, and the epidemic propagation model, all described in ???. This avoids any serialisation step between storage and computation: the router receives a Python object reference, runs the algorithm, and returns results without intermediate conversion.

Several routers maintain per-SBOM result caches at the module level. Community detection and k-core results are expensive to recompute for large graphs; caching ensures that repeated requests for the same SBOM complete in constant time. All caches are explicitly invalidated when an SBOM is updated or deleted, guaranteeing consistency between the stored graph and any cached analytical results.

4.1.4 Communication: REST API

The REST API is served at `http://localhost:8000` by default, overridable via an environment variable. CORS is configured to permit requests from the React development server. A health endpoint at `GET /health` returns a JSON object confirming the service status and version number; the frontend uses this to detect backend availability before enabling API-mode features.

The endpoints directly relevant to this chapter are: `POST /sboms` (file upload, graph construction, and storage), `GET /sboms/{id}` (retrieve a stored SBOM record), and `GET /sboms/{id}/metrics` (returns graph-level metrics including node count, edge count, average out-degree, and the five most-depended-on components ranked by in-degree). The remaining endpoints — `/vulnerabilities/{id}/impact`, `/vulnerabilities/{id}/audit`, `/vulnerabilities/{id}/graph-analysis`, `/vulnerabilities/{id}/kcore`, `/vulnerabilities/{id}/communities`, and `/vulnerabilities/{id}/epidemic` — are described in [Chapter 5](#) and ??? respectively.

4.2 CycloneDX Parsing Pipeline

Every SBOM in SBOM Lens enters through a single ingest endpoint and exits as a cached `nx.DiGraph` that all downstream routers query directly. The pipeline is designed for a “parse once, query many times” pattern: the graph is built on first ingest and never re-parsed unless the SBOM is explicitly updated. The three subsections trace each stage: upload and duplicate detection, component extraction and node creation, and dependency iteration and edge construction.

4.2.1 Ingestion and Deduplication

SBOM ingestion begins at the `POST /sboms` endpoint, which accepts a multipart file upload. The raw file bytes are read in full before any graph construction begins, allowing

the system to perform two correctness checks upfront.

The first check is JSON validation. The raw bytes are passed to `json.loads()`; if parsing fails, an HTTP 400 response is returned immediately. This guard ensures that no partially constructed graph can enter the store from a malformed file.

The second check is duplicate detection. A SHA-256 hash of the raw file bytes is computed with `hashlib.sha256(contents).hexdigest()` and compared against the "hash" field of every stored SBOM record. If a match is found, the endpoint returns `{"isDuplicate": True, "existingSBOM": existing}` without re-storing the file or reconstructing the graph. This “parse once, query many times” design avoids redundant computation and prevents double-counting in metrics when the same SBOM is submitted more than once.

A capacity limit prevents unbounded memory growth: the `MAX_SBOMS` environment variable (default: 10) caps the number of concurrently stored SBOMs. An upload that would exceed the limit is rejected with HTTP 409 Conflict. This limit is appropriate for a research prototype evaluated on a fixed dataset.

4.2.2 Component Extraction

Once the file passes validation and deduplication, the CycloneDX JSON structure is traversed to extract components. The root component — the project or application being described by the SBOM — is extracted from the `metadata.component` field and prepended to the `components[]` list. This placement gives the root a consistent position in the graph and allows the frontend to distinguish it visually as the primary node.

Each entry in the assembled component list is processed into a NetworkX node. The node identifier is resolved through a three-level fallback chain: `bom-ref` (the CycloneDX primary identifier) is used if present; otherwise `purl` (Package URL) is used; otherwise the component name. This chain mirrors the CycloneDX field hierarchy defined in Section 2.1 and ensures stable, unique node identifiers across SBOMs produced by different tools with varying levels of metadata completeness.

Node attributes are stored as the NetworkX node’s data dictionary. To avoid compatibility issues with NetworkX’s attribute handling, only scalar values — strings, integers, floats, and booleans — are retained from the component object. In practice, the persisted attributes are `name`, `version`, `purl`, `type`, and `bom-ref`. Components for which all three identifier fallbacks yield an empty or `None` value are silently skipped, making the parser tolerant of incomplete SBOMs encountered in the wild.

4.2.3 Graph Construction from Dependencies

After all nodes are created, the `dependencies[]` array is iterated to construct directed edges. Each entry in this array follows the CycloneDX dependency schema: a `ref` field identifying the depending component, and a `dependsOn` list identifying the components it depends upon. For each (`ref`, `target`) pair, a directed edge is added to the graph with `G.add_edge(ref, target)`.

The edge direction encodes a fundamental semantic: an edge from A to B means “A depends on B”. This directed convention underpins all downstream traversal algorithms. Blast-radius computation, for example, uses `nx.ancestors(G, node_id)` to find all components that transitively depend on a given node — ancestors in the directed graph are precisely those dependants, consistent with the `ref → target` edge direction.

Before adding each edge, a guard verifies that both `ref` and `target` already exist as nodes in the graph. Entries in `dependencies[]` that reference undeclared components are silently skipped. This tolerance is necessary because real-world SBOMs do not always maintain a complete correspondence between the `components[]` and `dependencies[]` sections.

The resulting `nx.DiGraph` is stored in `SBOMStore.graphs[sbom_id]` and never reparsed unless the SBOM is explicitly updated via `PUT /sboms/{id}`. All downstream routers retrieve the graph by calling `sbom_store.get_graph(sbom_id)`, receiving a direct reference to the live `DiGraph` object with zero serialisation overhead.

4.3 Graph Data Model

The directed graph constructed by the parsing pipeline (Section 4.2) encodes the SBOM’s component and dependency structure in a form that all downstream algorithms can consume directly. This section defines the node and edge data model — the representation used by all algorithms in Chapter 5 and ???. The in-memory store that manages graph lifecycle is described last.

4.3.1 Node Representation

Each node in the `DiGraph` represents one software component as declared in the CycloneDX SBOM. The graph contains exactly one node per distinct component identifier, as determined by the `bom-ref/purl/name` fallback chain described in Section 4.2.2. The node identifier — the key used to reference the node in all NetworkX operations — is a stable string assigned at ingest time and unchanged for the lifetime of the stored SBOM.

Each node carries a data dictionary with the following attributes:

- **name** — human-readable display label for the component.
- **version** — semantic version string as declared in the SBOM.
- **purl** — Package URL, used as the primary lookup key when matching components against vulnerability databases in Chapter 5.
- **type** — CycloneDX component type; one of `library`, `framework`, or `application`.
- **bom-ref** — the original CycloneDX `bom-ref` string, preserved for traceability back to the source file.

The root component (extracted from `metadata.component`) is a regular node in the NetworkX graph. The frontend distinguishes it visually by setting an `isPrimary` flag

in the Cytoscape node data object produced by `convertCycloneDXToGraph()`, which triggers a distinct green fill on the canvas.

4.3.2 Edge Representation

Each directed edge represents a direct dependency relationship between two components. An edge from node A to node B encodes the statement: *A directly depends on B*. This follows from the CycloneDX dependency schema, where an entry (`ref: A`, `dependsOn: [B, ...]`) produces the directed edge $A \rightarrow B$.

This directed semantics is load-bearing for every traversal algorithm in the system. Blast-radius analysis identifies all components that would be affected if a given component were compromised; these are its ancestors in the directed graph — the nodes from which there exists a directed path *to* the target, consistent with the $A \rightarrow B$ convention. Dominator-tree construction, depth histograms, and topological ordering all depend on the same directed convention, as described in [Chapter 5](#) and ??.

No attributes are stored on NetworkX edges. Edges are pure structural connectors: their presence encodes a dependency, and their direction encodes which component depends on which. On the frontend, each Cytoscape edge object carries an identifier formed from the source and target node IDs, a static label `depends_on`, and the CSS class `default-edge` for baseline styling.

4.3.3 In-Memory Store

The `SBOMStore` class manages the lifecycle of all stored SBOMs and their corresponding graphs. It maintains two parallel dictionaries: `self.sboms` maps each SBOM identifier to a full record dictionary (containing the SHA-256 hash, display name, creation timestamp, and the raw CycloneDX component and dependency lists); `self.graphs` maps the same identifier to the corresponding `nx.DiGraph`.

When a router needs to perform graph analysis, it retrieves the stored `DiGraph` via the store's `get_graph` method, which returns a direct Python reference to the live object. There is no serialisation step between storage and computation: the algorithm receives the graph object and operates on it in place. This design minimises latency for interactive analysis queries, where sub-second response times are necessary for a usable graph exploration interface.

Several routers maintain module-level caches for computationally expensive results — community detection and k-core decomposition are cached after the first request for a given SBOM. These caches are explicitly invalidated whenever an SBOM is updated or deleted, ensuring that stale results are never returned to the frontend.

As noted in [Section 4.1.1](#), the store's state is ephemeral: a server restart clears all SBOMs and graphs. This design is appropriate for the research context, and persistent storage is listed as a limitation and direction for future work in ??.

4.4 SBOM Graph Visualisation

The SBOM Graph is the primary interface through which the user explores the dependency structure of a loaded SBOM. This section describes the base rendering layer implemented with Cytoscape.js, the three available layout algorithms, and the core interaction model. Analysis overlays — vulnerability highlighting ([Chapter 5](#)) and network-science colouring (??) — extend the base layer described here.

4.4.1 Graph Rendering with Cytoscape.js

Cytoscape.js is initialised imperatively inside the `GraphVisualizer` React component using the `useRef` and `useEffect` hooks. The Cytoscape instance is stored in a ref rather than component state, so operations on the canvas — panning, zooming, applying a new layout — do not trigger React’s reconciliation cycle. This separation keeps the graph canvas responsive under interactive load.

The default visual encoding uses a consistent style throughout the application. Component nodes are rendered as circular shapes (28×28 pixels) filled with Material-UI’s info blue, with the component name displayed as a label below the node. The root component — the project whose dependencies are being analysed — is distinguished by a green fill and a 2-pixel border, making it immediately identifiable as the anchor of the dependency tree. Dependency edges are rendered as Bézier curves with triangle arrowheads at the target end, indicating the direction of dependency. Edges carry the static label `depends_on` with `auto-rotate` enabled so the label follows the curve.

Overlay CSS classes are pre-defined in the Cytoscape stylesheet and applied programmatically when an analysis mode is activated. The pre-defined overlay classes are: `.articulation-point` (red), `.blast-radius` (purple), `.dominator-tree` (cyan), `.vuln-node` (orange), `.diamond-node` (green), and `.hits-active` (node size and colour scaled from HITS score attributes). A `.dimmed` class reduces opacity to 15% to grey out nodes and edges that are not part of the current selection or overlay. The analytical meaning of each overlay class is defined in [Chapter 5](#) and ??; this section establishes the visual layer only.

4.4.2 Layout Algorithms

SBOM Lens exposes three Cytoscape.js layout algorithms, accessible via toolbar buttons, each suited to a different graph exploration objective.

The **Tree Layout** (Cytoscape name: `breadthfirst`, `directed: true`) organises nodes in horizontal levels according to their distance from the root in a breadth-first traversal. This layout makes dependency depth immediately visible: direct dependencies appear on level one, transitive dependencies on level two and beyond. It is the default layout for the Delta and Comparison views, where structural differences between two SBOMs are easier to compare when both graphs share the same hierarchical orientation.

The **Radial Layout** (Cytoscape name: `concentric`) places nodes in concentric rings around a central hub, with the root component at the centre. This is the default layout applied when a new SBOM is first loaded, giving the user an immediate overview of the dependency neighbourhood around the root.

The **Force-Directed Layout** (Cytoscape name: `cose`) simulates a physical spring system to distribute nodes so that connected components cluster together organically without imposing a hierarchical structure. This layout is particularly effective for revealing natural groupings in large, densely connected SBOMs.

All three layouts share common configuration options: `fit: true` (zoom to fit the graph on screen), a padding of 150 pixels, overlap avoidance, and an animated transition of 600 ms. Layout changes are applied imperatively by re-running the Cytoscape algorithm on the existing node and edge data without issuing a new request to the backend.

4.4.3 Core Interaction Model

The SBOM Graph supports the following core interactions, designed to enable focused exploration of the dependency structure.

Node selection. Tapping a node selects it; a blue border is applied to the highlighted node. Tapping the canvas background deselects all nodes.

Node inspection. Selecting a node opens a sidebar panel displaying the node's stored attributes: name, version, purl, type, and bom-ref. This panel gives the analyst the metadata needed to cross-reference the selected component with external vulnerability databases.

Zoom and fit. The current zoom level is tracked in React component state and displayed in the toolbar. A fit-to-view button resets the viewport so the entire graph is visible, which is useful after navigating to a deeply nested subgraph.

SBOM selection. A selector component lists all SBOMs currently stored in `SBOMStore`. Selecting an SBOM converts the stored CycloneDX data to Cytoscape node and edge objects and loads them into the Cytoscape instance.

Impact analysis entry point. Right-clicking a node opens a context menu that includes an impact analysis option, which fetches blast-radius and dominator-tree data from the backend. The analytical meaning of the resulting overlay is described in [Chapter 5](#); the interaction gesture is documented here as part of the interaction model.

Dynamic graph editing features — adding new nodes, repositioning nodes by dragging, and creating edges via the `cytoscape-edgehandles` plugin — are implemented in the application but are outside the scope of this thesis. These are graph construction utilities, not structural risk assessment features, and are not evaluated in this work.

5

Vulnerability Analysis

Chapter 4 established the directed dependency graph that SBOM Lens constructs from a CycloneDX SBOM. Every node in that graph carries a `pur1` attribute — a Package URL uniquely identifying the component — as defined in [Section 4.3.1](#). Chapter 5 uses those PURL attributes as the anchor for vulnerability matching: by querying three external vulnerability databases with the PURLs present in the graph, SBOM Lens enriches the structural model with real-world risk data. The three sources queried are the Open Source Vulnerability database (OSV), which acts as the primary automatic source, and the National Vulnerability Database (NVD) and GitHub Security Advisories (GHSA), both of which are architecturally integrated but gated behind API-key configuration. The chapter is organised into five sections. [Section 5.1](#) describes the rationale for using three sources and the integration pipeline for each. [Section 5.2](#) describes how vulnerability records are stored in the in-memory `VulnStore` and deduplicated by CVE ID and PURL pair. [Section 5.3](#) explains how CVSS v3.1 base scores are computed on the client side and maps scores to the four severity bands used throughout the system. [Section 5.4](#) covers the analyst-facing management workflow: automated fetching on SBOM load and the five-state status lifecycle exposed through the REST API. [Section 5.5](#) describes how vulnerability data is projected back onto the Cytoscape graph as three distinct overlay types — severity colour coding, blast-radius highlighting, and dominator-tree highlighting — giving analytical meaning to the CSS overlay classes introduced in [Section 4.4.1](#).

5.1 Multi-Source Vulnerability Integration

Matching SBOM components against known vulnerabilities requires querying external databases that maintain curated records of published CVEs. This section justifies the decision to query three databases rather than one, and describes the integration pipeline for each source.

5.1.1 Rationale for Multi-Source Design

No single vulnerability database covers all software ecosystems with equal completeness. NVD, maintained by NIST, catalogues vulnerabilities using the Common Platform Enumeration (CPE) naming scheme, which was designed for commercial software and is less well suited to the PURL-native package identifiers used by open-source registries. As noted in [Section 2.5.1](#), CPE strings can produce false positives for common package names because the matching is based on vendor and product strings rather than the precise package coordinate encoded in a PURL. OSV, by contrast, was designed from the outset to accept PURL queries directly, making it inherently more precise for open-source components. GHSA publishes its advisories in OSV format and feeds them into the OSV aggregation pipeline; however, newly published GHSA entries for npm and PyPI packages may precede their appearance in the OSV aggregation by hours to days, giving direct GHSA queries a recency advantage for those ecosystems.

The three-source design is therefore a recall-maximisation strategy. Different databases may carry the same CVE at different times, may assign different severity metadata to the same advisory, or may cover ecosystems that the others miss. By querying OSV, NVD, and GHSA independently and deduplicating on the CVE ID + PURL pair (described in [Section 5.2](#)), SBOM Lens obtains the broadest possible coverage without double-counting records.

OSV is designated as the primary source for three reasons: its `/v1/querybatch` endpoint accepts PURL-native queries, it aggregates advisories from more than fifty upstream sources, and it requires no API key — making it usable in any development or demonstration context without pre-registration. NVD and GHSA are architecturally present in the system and described in detail below, but both require API-key configuration. This asymmetry is transparent: examiners running SBOM Lens without configuring additional credentials will observe only OSV-derived vulnerability results, and this is exactly consistent with the framing presented in this chapter.

5.1.2 OSV: Primary Source

The OSV integration is implemented as a two-phase pipeline in `fetchVulnerabilitiesForSBOM()` inside `vulnerabilityAPI.js`. The first phase issues a single batch request to the `/v1/querybatch` endpoint with an array of PURL queries — one entry per component in the loaded SBOM. This batch design avoids N serial HTTP requests for an SBOM with N components: the entire component list is submitted in one round-trip, and the response returns the matching OSV vulnerability IDs for each PURL position.

The second phase resolves each unique OSV ID to its full advisory record by fetching `/v1/vulns/{id}`. The detail record contains the canonical summary, published and modified dates, a list of CVSS severity objects, and an `aliases` array that may include CVE identifiers. CVE alias resolution is performed by scanning the `aliases` array for

an entry beginning with the prefix "CVE-": the first such alias becomes the canonical `cveId` stored in `VulnStore`. If no CVE alias is present, the OSV identifier itself is used as the `cveId` fallback; deduplication logic (Section 5.2.2) still operates correctly on OSV IDs.

CVSS severity is extracted from the `severity[]` array in the detail record by selecting the entry whose `type` field is "CVSS_V3" or, for forward compatibility, "CVSS_V4". The `score` field of that entry is a CVSS vector string rather than a pre-computed numeric score; the numeric base score is therefore computed on the client by passing the vector to `cvssV3BaseScore()`, described in Section 5.3.3.

The pipeline is invoked automatically through the `useVulnerabilityFetcher` React hook whenever a new SBOM is selected in the frontend. The hook follows a fire-and-forget pattern: it is asynchronous, errors are caught and logged at the `console.warn` level, and the graph view is never blocked waiting for vulnerability data to arrive. Records are injected into `VulnStore` sequentially rather than in parallel: submitting all records in parallel would cause duplicate POSTs to race the backend deduplication check before it has processed the first record. On subsequent loads of the same SBOM, all duplicate CVE ID + PURL pairs are silently rejected by the backend with HTTP 409, and the frontend hook treats 409 as a no-op, so the analyst observes no duplication.

5.1.3 NVD Integration

The NVD integration queries the NVD API v2.0 endpoint `GET /rest/json/cves/2.0?keywordSearch={keyword}` with an `apiKey` HTTP header. The function `fetchFromNVD()` in `vulnerabilityAPI.js` implements this integration; it returns an empty array immediately if no API key is provided, making the integration strictly opt-in.

When an API key is configured, NVD returns CVE records with official CVSS v3.1 base scores sourced from `metrics.cvssMetricV31[0].cvssData.baseSeverity`, providing an authoritative severity reference that can be compared against the client-side computation described in Section 5.3.3. NVD results are also injected into `VulnStore` via the same `POST /vulnerabilities` endpoint used by the OSV pipeline, so records from both sources are managed uniformly.

A limitation of keyword-based NVD search is precision: querying by package name rather than PURL may return CVEs for similarly named but unrelated projects, a manifestation of the CPE matching ambiguity described in Section 2.5.1. The deduplication layer (Section 5.2) mitigates double-counting when the same CVE is returned by both OSV and NVD, but it cannot prevent the storage of false-positive NVD results if the keyword search matches an unrelated CPE entry.

5.1.4 GitHub Security Advisories

The GHSA integration uses the GitHub GraphQL API at `https://api.github.com/graphql` with an `Authorization: Bearer {token}` header. The function `fetchFromGitHub()`

returns an empty array if no token is provided; like the NVD integration, GHSA querying is therefore opt-in and requires explicit credential configuration.

As noted in [Section 2.5.3](#), GHSA publishes its advisories in OSV-compatible format and contributes them to the OSV aggregation pipeline. In practice, this means that most GHSA coverage is already subsumed by an OSV query. The value of a direct GHSA query lies in its recency: advisories for npm and PyPI packages may appear on GitHub Security Advisories before they have propagated through the OSV aggregation pipeline, giving the direct GHSA integration a window of coverage that OSV alone cannot provide. GHSA vulnerability identifiers use the format `GHSA-xxxx-xxxx-xxxx`; where a CVE cross-reference is available, it is used as the canonical `cveId`, preserving consistency with records sourced from OSV and NVD.

5.2 Deduplication and Storage

Querying three vulnerability databases for the same set of components inevitably produces overlapping results: OSV, NVD, and GHSA may each report the same CVE for the same component. `VulnStore` is the in-memory store that absorbs records from all three pipelines and ensures that each unique vulnerability-component pair is stored exactly once. This section describes the storage schema and the deduplication logic.

5.2.1 VulnStore Design

`VulnStore` is implemented as a Python class in `backend/store/vuln_store.py`. It maintains a single in-memory dictionary, `self.vulns`, keyed by UUID strings generated at record creation time with `str(uuid.uuid4())`. State is ephemeral: the dictionary is cleared on server restart, consistent with the ephemeral design of `SBOMStore` described in [Section 4.3.3](#) and appropriate for a research prototype evaluated on a fixed dataset.

Each record in `self.vulns` conforms to a consistent schema:

- **id** — UUID string uniquely identifying the record.
- **purl** — Package URL of the affected component, matching the node attribute described in [Section 4.3.1](#).
- **cveId** — CVE identifier or OSV identifier if no CVE alias was resolved.
- **severity** — severity label (CRITICAL, HIGH, MEDIUM, or LOW) derived from the CVSS base score.
- **status** — current remediation status; one of NOT_STARTED, IN_PROGRESS, RESOLVED, WONT_FIX, or FALSE_POSITIVE.
- **patchAvailable** — boolean indicating whether a patched version is known to exist.
- **notes** — free-text analyst annotation.
- **source** — originating pipeline: "osv", "nvd", "GitHub", or "Manual".
- **publishedDate** — date the advisory was first published.

- **lastUpdated** — server-side timestamp refreshed on each PUT update.
- **timeline** — append-only list of status-transition objects, each carrying a timestamp and the new status value, supporting an audit trail of remediation history.

The source field is particularly important in a multi-pipeline context: it preserves the provenance of each record, allowing an analyst to understand whether a vulnerability was identified by the automatic OSV scan, by a configured NVD or GHSA query, or by manual entry.

5.2.2 Deduplication Logic

Deduplication is enforced in the `VulnStore.add()` method before any new record is committed to the store. The dedup key is the pair of `cveId` and `purl`: the method searches the existing records for any entry where both fields match the incoming record.

```
duplicate = next((v for v in self.vulns.values()
                  if v.get("cveId") == cve_id and v.get("purl") == purl), None)
```

If a matching record is found, `add()` returns `None` without modifying the store. The REST endpoint wrapping this method responds with HTTP 409 Conflict. The `useVulnerabilityFetcher` hook treats a 409 response as a silent no-op: no error is raised, no duplicate appears in the store, and the analyst sees no change.

OSV IDs that could not be resolved to a CVE alias retain their OSV identifier as the `cveId` field; the deduplication logic operates on that OSV ID equally well, preventing duplicate OSV records in the rare case where the same OSV entry is fetched more than once.

Because the OSV pipeline is triggered automatically on every SBOM load and NVD and GHSA are triggered only when API keys are configured, OSV will typically win the first-write race for any vulnerability that all three sources carry. Cross-source severity merging is not implemented: the first-written record's severity is preserved, and subsequent additions for the same pair are rejected. The source field of the stored record identifies which pipeline contributed it, providing the provenance information needed to assess the severity metadata's origin.

5.3 CVSS v3.1 Base Score Computation

Vulnerability records from the OSV pipeline carry CVSS vector strings rather than pre-computed numeric scores. SBOM Lens computes base scores from these vectors entirely on the client side, with no dependency on an external CVSS scoring service. This section describes how the CVSS v3.1 formula is applied, illustrates the computation with the Log4Shell vector, and explains the design of the custom JavaScript implementation.

5.3.1 Formula and Metric Groups

The CVSS v3.1 base score formula and the numerical values for each metric are fully defined in [Section 2.5.4](#); this section does not repeat that derivation, but states explicitly the severity band thresholds that SBOM Lens uses to map numeric scores to visual encodings.

SBOM Lens maps CVSS base scores to four severity bands:

Severity	Score Range
CRITICAL	≥ 9.0
HIGH	7.0–8.9
MEDIUM	4.0–6.9
LOW	< 4.0

These thresholds follow the NVD/NIST severity rating scale. FIRST.org’s published five-band scheme adds a “None” band for a base score of exactly 0.0; SBOM Lens collapses this band into LOW to simplify the overlay colour mapping to four distinct visual classes. The `parseCVSSSeverity()` function also handles NaN and undefined inputs by returning LOW as a safe default, ensuring that components with incomplete severity metadata are never rendered with a misleadingly elevated colour.

The scope-sensitivity of the Privileges Required (PR) metric is handled explicitly: when Scope is Changed (S:C), the PR Low coefficient is 0.68 rather than 0.62, and PR Medium is 0.50 rather than 0.68. The custom implementation branches on the parsed Scope value when looking up PR coefficients, matching the FIRST.org specification precisely.

5.3.2 Worked Example: Log4Shell

Log4Shell (CVE-2021-44228) was introduced as the principal illustrative case study in [Section 2.5.4](#) and analysed from a supply-chain perspective in [Section 3.3.2](#). Chapter 5 applies the same vulnerability to the implementation: given its CVSS vector, the client-side scoring function must produce the expected base score.

The Log4Shell vector is:

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

Substituting the numerical values specified in [Section 2.5.4](#) gives: $AV = 0.85$, $AC = 0.77$, $PR = 0.85$ (Scope Changed branch), $UI = 0.85$, $C = 0.56$, $I = 0.56$, $A = 0.56$. The Impact Sub-Score (ISS) is:

$$ISS = 1 - (1 - 0.56)(1 - 0.56)(1 - 0.56) = 1 - 0.44^3 = 1 - 0.0785 \approx 0.922$$

Because Scope is Changed, the Impact term uses the modified formula:

$$\begin{aligned} \text{Impact} &= 7.52 \times (ISS - 0.029) - 3.25 \times (ISS - 0.02)^{15} \\ &\approx 7.52 \times 0.893 - 3.25 \times 0.893^{15} \\ &\approx 6.715 + 2.389 \approx 9.104 \end{aligned}$$

The Exploitability term is:

$$\text{Exploitability} = 8.22 \times 0.85 \times 0.77 \times 0.85 \times 0.85 \approx 4.833$$

The base score is therefore:

$$\begin{aligned} \text{Base} &= \min(1.08 \times (9.104 + 4.833), 10) \\ &= \min(1.08 \times 13.937, 10) \\ &= \min(15.05, 10) = 10.0 \end{aligned}$$

The CVSS roundup rule produces a final score of 10.0, which falls in the `CRITICAL` band (≥ 9.0). In the graph overlay described in [Section 5.5.1](#), a Log4Shell-affected component node is therefore assigned the `.vuln-critical` CSS class and rendered in red.

5.3.3 Custom Client-Side Implementation

The CVSS scoring logic is implemented entirely in `vulnerabilityAPI.js` with no external CVSS library dependency. This is a deliberate design choice: adding a CVSS computation library to the frontend `package.json` would introduce a runtime dependency whose licensing, maintenance status, and specification compliance would need independent verification. Implementing the formula from the FIRST.org specification in approximately seventy lines of JavaScript keeps the frontend self-contained and makes the scoring logic directly auditable alongside the rest of the source code.

The function `cvssV3BaseScore(vector)` accepts a CVSS vector string, splits it on the `"/"` separator to extract metric abbreviation–value pairs, and looks each pair up in a coefficient table to produce the numeric values used in the formulas above. It branches on the parsed Scope value to select the correct Impact and PR coefficients, computes the ISS, Impact, Exploitability, and Base sub-terms in sequence, and applies the CVSS roundup rule (`Math.ceil(base * 10) / 10`) before returning the final score. This roundup rule is specified by FIRST.org as “the smallest number, specified to one decimal place, that is equal to or higher than its input”; the expression `Math.ceil(x * 10) / 10` implements it exactly.

The entry point for callers is `parseCVSSSeverity(scoreString)`, which accepts either a numeric score string (e.g. `"9.8"` as returned by NVD) or a CVSS vector string (as returned by OSV). If the input contains a `"/"` character it is treated as a vector and passed to `cvssV3BaseScore()`; otherwise it is parsed as a floating-point number directly. This unified entry point means the rest of the codebase does not need to handle

the NVD-numeric versus OSV-vector distinction explicitly.

5.4 Vulnerability Management Workflow

Beyond recording which components carry known CVEs, SBOM Lens provides a lightweight remediation-tracking workflow. Analysts can update the status of each vulnerability record, add notes, and observe how the record's status has changed over time. This section describes the automated fetch that populates the store on SBOM load and the REST API that exposes the status lifecycle.

5.4.1 Automated Fetch on SBOM Load

Vulnerability fetching is triggered automatically every time an SBOM is selected in the frontend, without requiring any analyst action. The `useVulnerabilityFetcher` React hook receives the `components []` array from the loaded SBOM and passes the PURL list to the OSV pipeline described in [Section 5.1.2](#).

The hook follows the fire-and-forget pattern introduced in [Section 5.1.2](#): it is asynchronous, does not block rendering, and swallows errors silently so that a network failure or an unresponsive vulnerability database never prevents the analyst from viewing the dependency graph. Records are submitted to the backend's `POST /vulnerabilities` endpoint one at a time rather than in a batch request; this sequential submission prevents a parallel flood of identical records from racing the deduplication check on the server side.

When the same SBOM is loaded a second time (or when the page is refreshed), the hook runs again and resubmits every PURL in the component list. All CVE ID + PURL pairs that already exist in `VulnStore` receive a 409 Conflict response; the hook treats each 409 as a no-op. The analyst therefore observes no change to the vulnerability dashboard on repeated loads.

5.4.2 Status Lifecycle and REST API

`VulnStore` exposes a full CRUD REST API through the `vulnerabilities` FastAPI router. The endpoints are:

- `GET /vulnerabilities` — returns all records in the store as a JSON array; no SBOM-scoping filter is applied, making the store a global view of all known vulnerabilities across all loaded SBOMs.
- `POST /vulnerabilities` — creates a new record; returns HTTP 201 on success or HTTP 409 if the CVE ID + PURL pair already exists.
- `PUT /vulnerabilities/{id}` — partial update of an existing record; the analyst may update status, notes, or `patchAvailable`; the `lastUpdated` timestamp is refreshed automatically by the server.
- `DELETE /vulnerabilities/{id}` — removes a single record.

- `DELETE /vulnerabilities` — clears the entire store; useful for resetting state between evaluation runs without restarting the server.

The `status` field supports a five-state remediation lifecycle. `NOT_STARTED` is the initial state assigned to every newly created record. `IN_PROGRESS` indicates that a remediation effort is under way. `RESOLVED` is the terminal success state, indicating that the vulnerability has been patched or mitigated. `WONT_FIX` records a deliberate decision not to remediate, for example because the vulnerable code path is not reachable in the deployment context. `FALSE_POSITIVE` records a triage conclusion that the reported vulnerability does not actually affect the component as used. Each status transition is appended to the `timeline` list with a server-side timestamp, creating an append-only audit trail of the remediation history for each record.

5.5 Graph Overlay Visualisation

The vulnerability data held in `VulnStore` is projected back onto the `Cytoscape.js` graph through three distinct overlay types. Each overlay assigns one of the CSS classes predefined in [Section 4.4.1](#) — `.vuln-critical`, `.vuln-high`, `.vuln-medium`, `.vuln-low`, `.blast-radius`, `.dominator-tree`, and `.dimmed` — to the appropriate nodes, giving those classes the analytical meaning that Chapter 4 deferred to this chapter. The passive severity colour-coding overlay is active whenever vulnerabilities are loaded; the blast-radius and dominator-tree overlays are triggered interactively by the analyst.

5.5.1 Severity Colour Coding

The severity colour-coding overlay is a passive, always-on projection. Whenever the vulnerability dashboard is active and `VulnStore` contains records, each graph node is assigned a CSS overlay class determined by the highest-severity CVE that affects the component identified by that node's PURL.

A component may carry multiple CVE records in `VulnStore`; in that case the highest severity wins. A node whose PURL is associated with one `CRITICAL` and three `MEDIUM` vulnerabilities is displayed as `CRITICAL` — the most dangerous exposure takes visual precedence. The Cytoscape overlay is applied programmatically: for each node, the vulnerability records whose `purl` field matches the node's PURL are retrieved from the store, the maximum severity is computed using `parseCVSSSeverity()`, and `node.addClass('vuln-' + severity.toLowerCase())` assigns the appropriate CSS class. Nodes whose PURL matches no stored vulnerability record remain in the default info-blue style; the absence of an overlay class thus encodes unaffected status without requiring any explicit negative marking. [Figure 5.1](#) shows the overlay applied to a loaded SBOM.

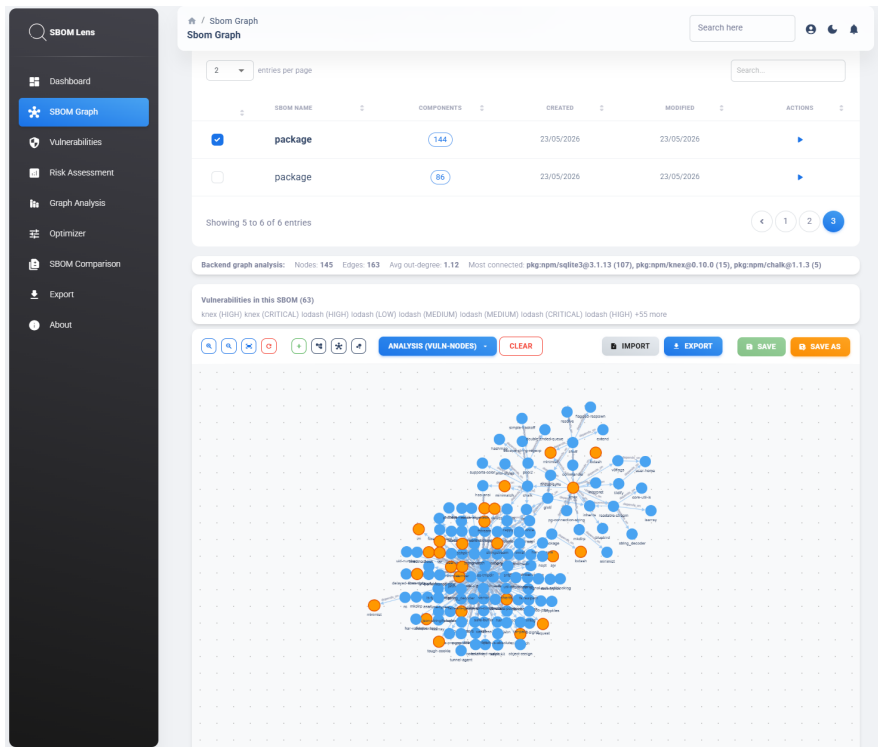


Figure 5.1: SBOM Lens graph overlay showing vulnerability severity colour coding. Nodes are coloured by their highest-severity CVE: red for CRITICAL, orange for HIGH, yellow for MEDIUM, and green-yellow for LOW. Unaffected nodes retain the default blue style.

5.5.2 Blast Radius Analysis

The blast-radius overlay is an interactive analysis triggered when the analyst selects a node and requests impact analysis from the context menu, as described in [Section 4.4.3](#). The query is issued to GET `/sbom_id/impact?node_id={id}`, which returns the blast radius, dependency paths, dominated nodes, and per-node HITS scores in a single response.

On the backend, the blast radius is computed as `list(nx.ancestors(G, node_id)) + [node_id]`. The semantic interpretation follows directly from the edge direction established in [Section 4.3.2](#): an edge from A to B means *A depends on B*, so `nx.ancestors(G, B)` returns all nodes from which there exists a directed path *to* B — the upstream consumers of B whose security posture is degraded if B is compromised. This framing is intentional and distinct from a “downstream” interpretation: in the SBOM Lens graph, the upstream consumers are the packages that *pull in* the vulnerable component, and they are precisely the packages that an attacker can exploit by compromising the vulnerable node.

The API response also includes up to five shortest paths from the root packages (components with in-degree zero) to the vulnerable node, computed with `nx.all_shortest_paths` from each root. These paths provide concrete dependency chains — a sequence of “package X uses package Y which uses the vulnerable package Z” — that an analyst can use to understand exactly how the vulnerable component entered the dependency

tree.

On the frontend, blast-radius nodes receive the `.blast-radius` CSS class (purple, as pre-defined in [Section 4.4.1](#)), and all other nodes are assigned the `.dimmed` class, reducing their opacity to 15 % and focusing the analyst's attention on the affected portion of the graph.

The impact API response also includes `hitsHub` and `hitsAuthority` scores for each node in the blast radius. These scores quantify the structural centrality of each node in the dependency network and are relevant to prioritising remediation effort; however, the HITS algorithm that produces them is described in detail in ?? (specifically ??) and is not explained further here.

5.5.3 Dominator Tree Analysis

The dominator-tree overlay is returned in the same `GET /{sbom_id}/impact` response as the blast radius, adding structural risk information without requiring an additional round-trip.

On the backend, `nx.immediate_dominators(G, node_id)` is called with the vulnerable node as the root; the function returns a dictionary mapping each graph node to its immediate dominator. The dominated set consists of all nodes whose immediate dominator is the vulnerable node itself: these are the components for which no alternative path to the graph root exists that bypasses the vulnerable node. In supply-chain terms, a node that dominates others is a structural chokepoint: if it were removed or compromised, all the dominated components would lose their connection to the root of the dependency tree with no alternative route.

Dominated nodes are assigned the `.dominator-tree` CSS class on the frontend, displayed in cyan as pre-defined in [Section 4.4.1](#). Because the dominator-tree overlay is rendered simultaneously with the blast radius within the same interaction, the analyst can read both overlays together: nodes highlighted in purple are the upstream consumers affected by the vulnerability's propagation, while nodes highlighted in cyan are structurally dependent on the vulnerable node and would be isolated if it were removed. This dual encoding gives both a security-propagation view and an architectural-risk view in a single inspection, providing complementary information that neither overlay conveys alone.

Appendices

Annexes

